



SECP2523: DATABASE

SEM I 2024/2025

DATABASE DESIGN DESCRIPTIONS: LOGICAL DATABASE DESIGN FOR KADA Online System

Group Name: Technicrab

Members:

1. Cheryl Cheong Kah Voon A23CS0060
2. Chau Ying Jia A23CS0213
3. Neo Li Xin A23CS0253
4. Lau Yee Wen A23CS0099
5. Woo Cheng Shuan A23CS0283

Stakeholder: Lembaga Kemajuan Pertanian Kemubu (KADA)

Printing Date: 27 January 2025

TABLE OF CONTENTS

CHAPTER	TOPIC		PAGE
1	Introduction		3
	1.1	Overview about the company < Provide the overview of the company >	3
	1.2	The Existing System < Provide the description of the current system (The description shall consist, among others, of what the system does, is it a manual or automated system and the problem(s) faced by the client (users) of the system. >	3
	1.3	The Proposed System < Description of the proposed system (i.e. the to-be system) definition. The description shall consist, among others, the mission statement of the to-be system, the overview of what the to-be system will do, the mission objectives, the system scope and boundaries with the Use Case Diagram, and the proposed technology to be used in designing and developing the system. >	8
2	Global Conceptual Data Model (Global ERD) < This is the ERD as a result of merging the local data models >		11
	2.1	Global Conceptual Database Design (Conceptual ERD)	11
	2.2	Data Dictionary for the global conceptual ERD.	12
3	Logical Data Model (Logical ERD) < This is the ERD as a result of transforming the global conceptual data models >		20
	3.1	Logical Database Design (Logical ERD)	20
	3.2	Data Dictionary for the Logical ERD < Update the data dictionary based on the normalized relations produced from logical ERD >	20
	3.3	Relational Database Schema(s) < Derive relation schema(s) from the logical ERD >	24
	3.4	Normalization	27

		<i>< Perform normalization up till 3NF ></i>	
4		Interface Design and SQL Implementation	41
	4.1	Interface Design (Mapped to the Proposed SQL Statement)	41
	4.2	SQL Statements (DDL & DML)	55
5		Summary	74
6		Appendices	75
	6.1	Use Case Diagram for the system.	75
	6.2	Activity Diagrams for every use case identified.	75
	6.3	Meeting Log	76

NOTE: A module can consist of one or multiple (related) use case. A team member in a group must be responsible to at least one module.

1. Introduction

1.1 Overview about the company

Koperasi Kakitangan KADA Kelantan Berhad, a subsidiary of Lembaga Kemajuan Pertanian Kemubu (KADA) under the Ministry of Agriculture and Food Security, aims to improve the economic and social well-being of farmers in the Kemubu region. The cooperative provides its members with essential financial and administrative services, as well as the tools and resources they need to grow sustainably.

To streamline operations and improve efficiency, the cooperative intends to move away from its current manual processes and towards a comprehensive digital platform tailored to its organisational goals.

1.2 The existing system

The existing system operates manually, which limits its efficiency and productivity. Nevertheless, it does offer several main services and features, as listed below:

1. Membership Registration

This is a structured process for KADA's staffs to become part of their members, granting them accessible to a variety of cooperative benefits and services.

The current member registration system relies on a manual process, necessitating individuals to complete information on printed forms. During registration, various personal details, such as name, identification number, address, and others are gathered. Afterward, KADA staff will need to review the registration documents again. This method is not only time-consuming but also susceptible to errors and places a significant strain on human resources.

Figure 3.1 printed Member Registration Form

2. Loan Applications

Their loan application seeks financial assistance for personal, educational, or business needs. The goal of the loan application process is to be clear and efficient, allowing cooperative members to obtain the required funds quickly. Currently, the loan application system is manual too, necessitating that information be entered on printed forms. In addition, the qualified members are not able to check on their past transactions and records that is hard for them to look back for their total loans and payments if they lost their statements.

KOPERASI KAKITANGAN KADA KELANTAN BERHAD

PEMINJAMAN PEMAHAMAN ANGGOTA

DATA PERSONAL

☐ Laki-laki ☐ Perempuan
☐ Bekerja ☐ Tidak Bekerja
☐ Berkahwin ☐ Tidak Berkahwin

Nama Penuh: _____
 No. Kad: _____
 No. Telefon: _____
 Alamat: _____
 Poskod: _____

DATA KOPERASI

No. Anggota: _____
 No. Saham: _____
 No. Simpanan: _____
 No. Pinjaman: _____
 No. Lain-lain: _____

PERMOHONAN PINJAMAN

Nama Pinjaman: _____
 Jumlah Pinjaman: _____
 Tempoh Pinjaman: _____
 Tujuan Pinjaman: _____
 No. Lain-lain: _____

PERNYATAAN

Saya yang bertandatangan di bawah ini menyatakan bahawa saya adalah ahli Koperasi Kakitangan KADA Kelantan Berhad dan saya ingin memohon pinjaman. Saya bersetia untuk membayar balik pinjaman tersebut mengikut tempoh dan kadar yang ditetapkan.

Nama: _____
 No. Kad: _____
 No. Telefon: _____
 Alamat: _____
 Poskod: _____

PENYATAAN KOPERASI

Nama Koperasi: _____
 No. Pendaftaran: _____
 No. Saham: _____
 No. Simpanan: _____
 No. Pinjaman: _____
 No. Lain-lain: _____

REKOD PINJAMAN

No.	Nama	Jumlah	Tempoh	Tujuan	Status

PERMOHONAN PINJAMAN

Nama Pinjaman: _____
 Jumlah Pinjaman: _____
 Tempoh Pinjaman: _____
 Tujuan Pinjaman: _____
 No. Lain-lain: _____

Figure 3.2 Member Loan Applications Form

3. Member Financial Statements

Koperasi Kakitangan KADA Kelantan Berhad currently uses a manual system for generating and distributing member financial statements. This involves collecting financial data manually from various records and compiling it into individual member statements. These statements include details such as the member's name, IC number, shares held, loans taken, and savings.

Once the statements are prepared, they are reviewed by staff to ensure accuracy. After verification, the statements are handed out to members. Members then sign and return the verification portion to confirm receipt and accuracy.

This manual process is time-consuming, labor-intensive, and prone to errors due to manual data entry. It also places a significant burden on the cooperative's limited human resources and delays members' access to their financial information. Implementing an automated system would streamline this process, reduce staff workload, and provide members with timely and accurate financial information.

Figure 3.3 Member Financial Statement

Problem with Existing System

The registration system at KADA company is currently handled manually, showing signs of low efficiency and productivity, as described below. Even though it offers only a limited number of essential services and features, there is room for improvement. Recognizing the constraints of the existing system emphasizes the necessity for a complete overhaul to enhance the system. Moving towards a new computerized system could greatly elevate their services and features. This new system will bring about automation, streamlined processes, an improved website for better performance, and an expanded array of services for users.

1. Insufficient registration process

In the current system, all registration procedures are manually processed, like customers filling out paper forms to register and staff handling each form individually. Personal information, both private and sensitive, is predominantly stored in hard copies during registration. This method not only poses security risks but also makes it challenging to retrieve or locate information when necessary. Additionally, the registration papers require further collection, rearrangement and scrutiny by staff at the end. It really takes much more time than doing the registration online.

2. Inconvenient Status Checking

In the existing process, members are unable to check the status directly, they must either call or wait for an email from the company to obtain the current status, whether it is about the registration approval or payment status. This method is not that convenient, as staff have to manually send emails instead of having an automated system on their website that segregates the status. Improvements are necessary to modernize this outdated system to enhance the efficiency of the processes.

3. Heavy Dependence on Human Resources

The existing manual system faces an issue as it heavily depends on human resources. All the steps starting from the registration and application for the loan are done manually. If the number of human resources decreases beyond a certain point, it will impact productivity and efficiency in the workplace. To address the issue of heavy dependence on human resources, the implementation of automation solutions is crucial as it can help streamline processes, minimise errors, and free up employees to focus on more strategic tasks that require human intervention.

4. Inefficient and human error

We know that retrieving and managing physical documents is time-consuming and the manual system heavily depends on human resources. This might lead to the chances of human error rising when staff forget to email the current status, missing payment receipts, or failing to gather customer information and also missing information. These inefficiencies will cause delays in service delivery and mistakes in data processing, affecting overall customer satisfaction and also might disrupt the entire development process as incomplete or inaccurate data may result. Therefore, the company requires a more well-designed system.

5. Security Concerns of Manually Stored Customer Forms and Information

In the digital age, safeguarding customer information is paramount for maintaining trust and ensuring compliance with data protection regulations. The cooperative currently relies on manual storage of customer forms and information in physical cupboards. This method poses significant security risks that need urgent attention to protect sensitive data and uphold our commitment to customer privacy. As we can see, employees, cleaning staff, and visitors may easily access the cupboards, potentially leading to data breaches. Furthermore, it is challenging to track who accessed or modified the physical documents, making it difficult to audit and ensure the accountability of data handling practices.

Features	Existing System	Proposed System
Membership registration	Yes	Yes
Loan application	Yes	Yes

Application status checking	No	Yes
Check financial status and statements	Yes	Yes
Check monthly and annually report	No	Yes
Check profit and loss from account	No	Yes

1.3 The proposed system

The proposed Koperasi KADA Online System is a fully integrated digital platform aimed at enhancing the efficiency, security, and overall user experience of Koperasi Kakitangan KADA Kelantan Berhad. It replaces the current manual processes, which are time-consuming, prone to errors, and heavily reliant on human resources.

The system allows new users to register online by submitting personal details such as name, address, and identification data. Upon submission, this data is processed automatically, verified for accuracy, and stored securely in a digital database, and provides real-time feedback on the status of the application, whether it is successful, pending, or rejected. Once a user becomes a registered member, they gain access to several convenient services, such as checking the real-time status of their registration, loan applications, and payments through an online portal. Members can also manage their savings, view financial statements, and track potential dividends from their investments, receiving automated email or SMS notifications regarding their financial activities.

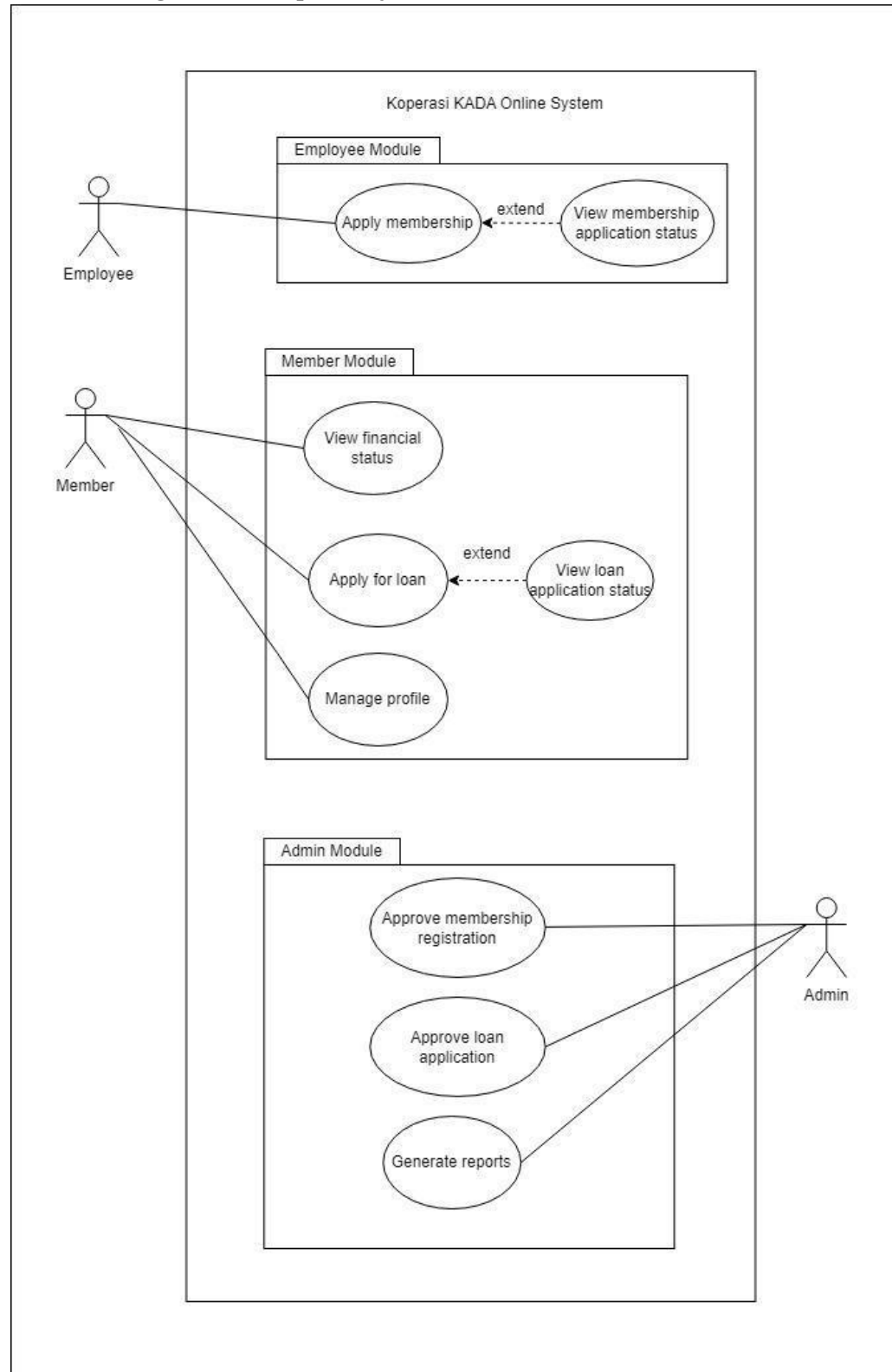
For the administrative staff (KADA employees), the system reduces the burden of manual data handling and processing. Admin are responsible for reviewing member applications, verifying submitted information, and authorizing loan applications based on pre-set criteria. For the admin, the system simplifies tasks by providing a centralized platform for reviewing member applications and loan requests. Admins can verify application details, approve or reject submissions based on eligibility criteria, and manage the digital records securely, reducing the need for paper-based processes. They can also update the status of applications and ensure that members are promptly notified of any changes. The system minimizes human error by automating many of the routine processes, freeing up staff to focus on more strategic and high-priority tasks.

Board Directors, who oversee high-level decisions, are involved in reviewing and approving larger or more critical loan applications. Their role is to ensure that these applications align with the cooperative's strategic goals and regulatory requirements. They have the authority

to make final decisions on both member and loan applications, with their approval or rejection logged into the system for transparency and future reference.

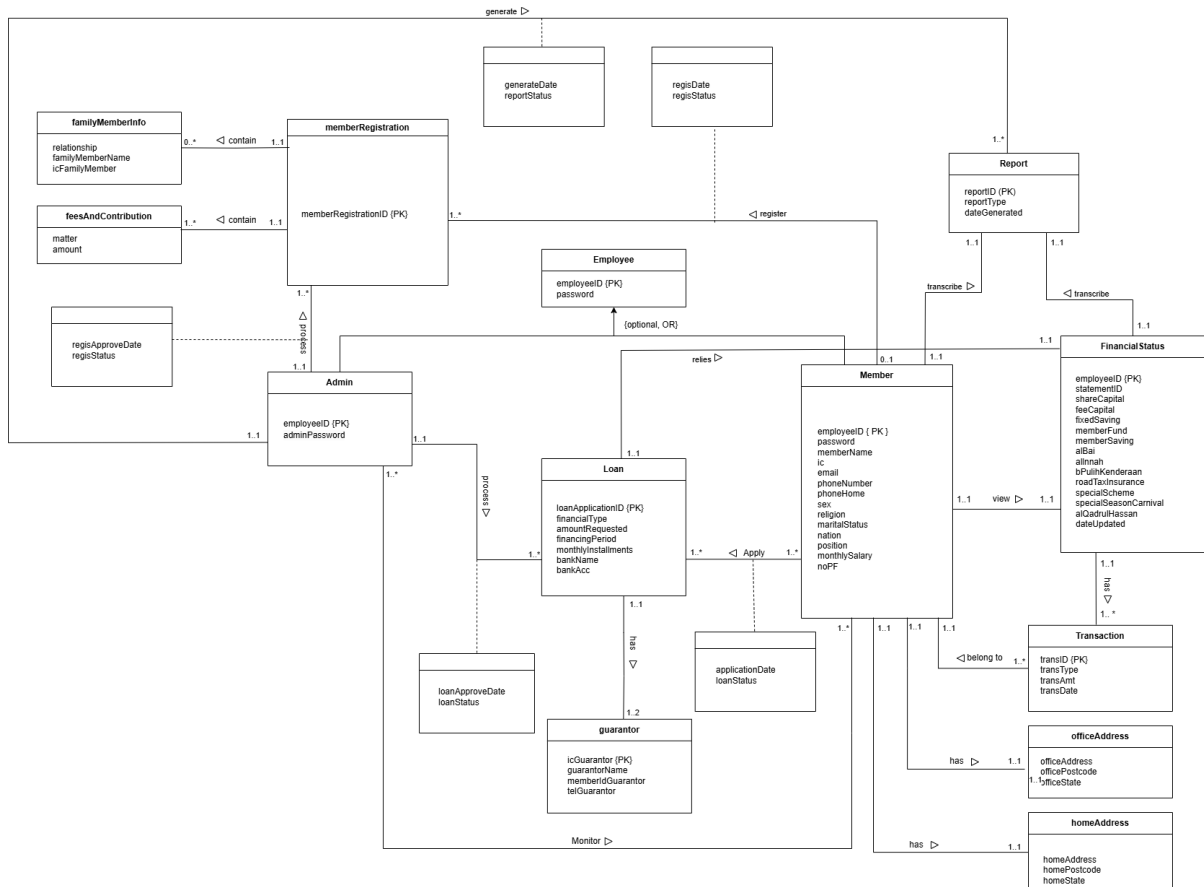
Overall, the KADA Registration System brings about a more efficient, secure, and user-friendly environment for all stakeholders. It reduces the burden on human resources, improves data management, enhances communication, and ensures that sensitive member information is stored securely. Also, it enhances security measures through features like Two-Factor Authentication (2FA), and providing a seamless experience for users. By automating key functions and providing a streamlined user interface, the system aims to elevate the cooperative's service delivery and operational capabilities, fostering greater member satisfaction and organizational productivity.

Use Case Diagram for Proposed System



2. Global Conceptual Data Model (Global ERD)

2.1 Global Conceptual Database Design (Conceptual ERD)



2.2 Data Dictionary for the global conceptual ERD.

2.2.1 Identifying Entity

Entity Name	Description	Aliases	Occurrence
employee	Represents any person accessing the KADA system	New user, guest	Occurs when first-time user visit the website
member	Represents registered users who have created KADA accounts	Members	Occurs when new user or guest register as a member
admin	Represents the system admin who responsible for managing KADA system	Administrator, staff, board director	Occurs when system built and processing members' account activity
memberRegistration	Describe all activities when a user register as a member	Users	Occurs when a new user register as a member through KADA system
familyMemberInfo	Represents all the information about members' family members recorded	Members' family record	Occurs when user register as a member
feesAndContribution	Represents fees paid by members and any contributions made towards KADA	Contribution and fees record	Occurs when user register as a member
loan	Represents loan applications submitted by members	Loan application record	Occurs when member apply for loan
guarantorDetails	Represents information about members' loan guarantors	Members' loan application guarantor details	Occurs when member apply for loan
financialStatus	Represents member's financial health	Members' financial reports / records	Occurs when member make a loan application and to check financial status
transaction	Represents all financial transactions made by members	Accounts loan / fees transaction	Occurs when admin make an report to be viewed by members
report	Stores generated reports related to financial summaries, transaction	Monthly / yearly reports, financial reports	Occurs when a monthly or yearly report have to

	histories, or member activities		be published to the members by admin
officeAddress	Stores members' office address	Members' personal information	Occurs when employer register as a member
homeAddress	Stores members' house address	Members' personal information	Occurs when employer register as a member

2.2.2 Identifying Relationship Types

Entity Name	Multiplicity	Relationship	Entity Name	Multiplicity
member	0..1	Registers	memberRegistration	1..1
	1..1	Has	homeAddress	1..1
	1..1	Has	officeAddress	1..1
	1..1	view	financialStatus	1..1
	1..*	Applies	loan	1..*
admin	1..1	approves	memberRegistration	1..*
	1..*	generate	report	1..1
	1..*	Monitor	member	1..*
	1..1	process	loan	1..*
memberRegistration	1..1	has	familyMemberInfo	0..*
	1..1	has	feesAndContribution	1..*
loan	1..1	Has	guarantorDetails	1..2
	1..1	relies	financialDetails	1..1
financialStatus	1..1	has	transaction	1..*
transaction	1..*	belong to	member	1..1

2.2.3 Attributes Description

Entity Name	Attributes	Data Type & Length	Description	Nulls	Multi-valued	Example
employee	employeeID(PK)	INT(4)	Unique ID for the employee	No	No	0001
	password	VARCHAR (30)	User's account password	No	No	P12345W
member	employeeID (PK)	INT(4)	Unique ID for member	No	No	0001
	password	VARCHAR (30)	Password for member login	No	No	P12345W
	memberName	VARCHAR (80)	Name of the member	No	No	Ali Abu
	ic	CHAR (12)	Identification number of the member	No	No	040101081234
	email	VARCHAR (30)	Member's email address	No	No	member@gmail.com
	phoneNumber	VARCHAR (15)	Contact number of the member	No	No	+60123456789
	sex	CHAR (1)	Gender of the member (M/F)	No	No	Female
	religion	VARCHAR (30)	Religion of the member	Yes	No	Buddha
	maritalStatus	VARCHAR (15)	Marital status	Yes	No	Single
	nation	VARCHAR (30)	Nationality of the member	Yes	No	Malaysia
	position	VARCHAR (30)	Working position of the member	No	No	Data Engineer
	monthlySalary	DECIMAL (10,2)	Member's monthly salary	No	No	10000.00

	noPF	INT(4)	Member no PF	No	No	5678
admin	employeeID (PK)	INT(4)	Unique ID for admin	No	No	4567
	adminPassword	VARCHAR (20)	Password of the admin	No	No	noahlim
memberRegistration	memberRegistrationID (PK)	VARCHAR (20)	Unique ID for member registration	No	No	MR123456789
familyMemberInfo	relationship	VARCHAR (30)	Relationship between family member and member	Yes	No	Daughter
	familyMemberName	VARCHAR (80)	Name of the family member	No	No	HenryWong
	icFamilyMember	CHAR (12)	Identification number of the family member	No	No	040101084567
feesAndContribution	entryFee	INT(4)	Entry fee	No	No	200.00
	modalShare	INT(11)	Modal share for member	No	No	200.00
	feeCapital	INT(11)	Member's fee capital	No	No	200.00
	deposit	INT(11)	Member's deposit	No	No	200.00
	contribution	INT(11)	Member's contribution	No	No	200.00
	fixedDeposit	INT(11)	Member's fixed deposit	No	No	200.00
	others	INT(11)	Others contribution	No	No	200.00

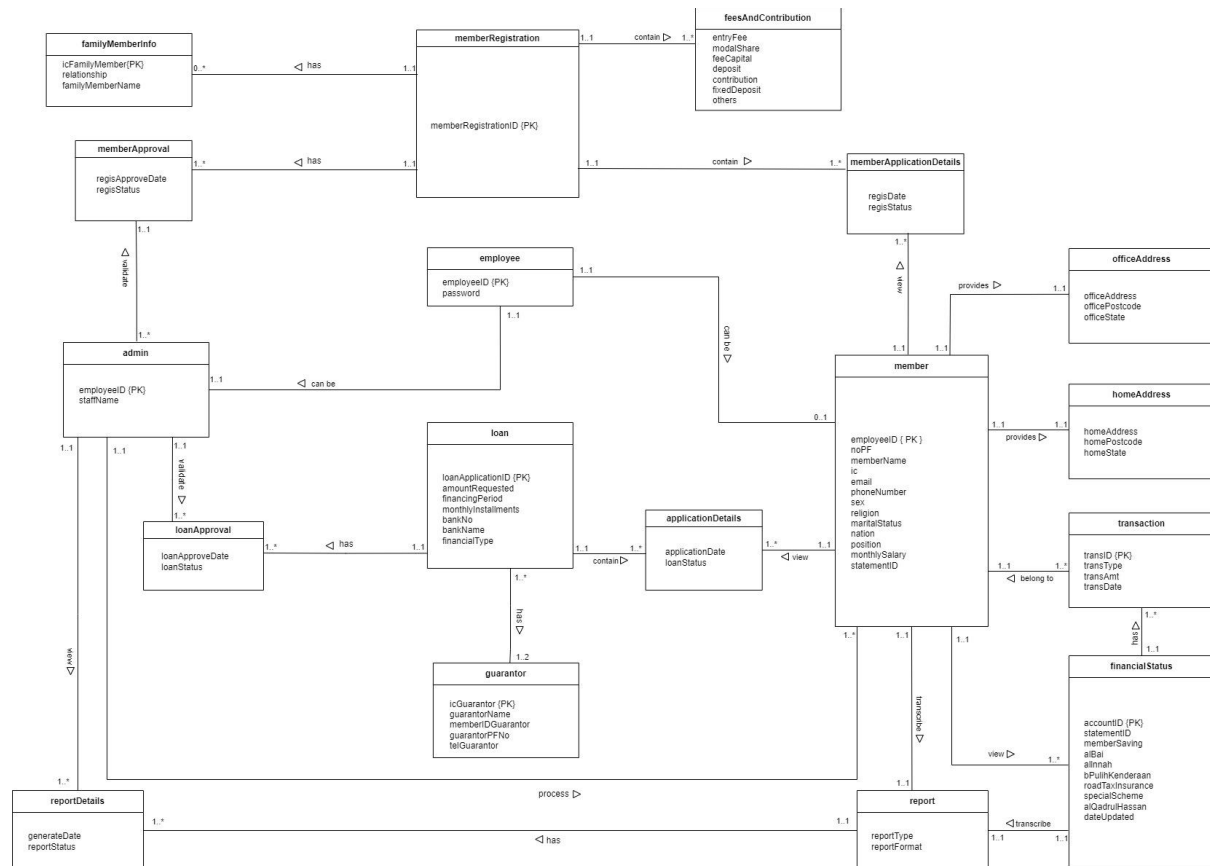
loan	loanType	VARCHAR(30)	Type of loan	No	No	AL_BAI
	loanApplication ID (PK)	VARCHAR (20)	Unique ID for loan application	No	AUTO-INCREMENT	7777
	amountRequested	INT (12,2)	Amount requested for the loan	No	No	30000.00
	financingPeriod	INT(11)	Loan repayment period in months	No	No	24
	monthlyInstallments	DECIMAL (10,2)	Monthly repayment amount	No	No	650.00
	bankName	VARCHAR (50)	Bank name for loan transfer	No	No	RHB
	bankAccount	VARCHAR (20)	Bank account number	No	No	01234567890
guarantor Details	guarantorName (PK)	VARCHAR (80)	Name of the guarantor	No	No	Evelyn Lee
	icGuarantor	CHAR (12)	Identification number of the member	No	No	040101081234
	memberIDGuarantor	VARCHAR (8)	Unique ID for member's guarantor	No	No	8888
	telGuarantor	VARCHAR (15)	Contact number of the member's guarantor	No	No	0123456789
financialS tatus	employeeID (PK)	INT(4)	Unique ID for employee	No	No	5522
	statementID	INT (20)	Unique ID for financial statement	No	No	12345678901

shareCapital	DECIMAL (12,2)	Total share capital for the member	No	No	2000.00
feeCapital	DECIMAL (10,2)	Total fee capital for the member	No	No	300.00
fixedSaving	DECIMAL (10,2)	Total fixed savings	No	No	1000.00
memberFund	DECIMAL (10,2)	Total member fund contribution	No	No	500.00
memberSaving	DECIMAL (10,2)	Total savings for the member	No	No	1500.00
alBai	DECIMAL (10, 2)	Total loan amount of Al-Bai	Yes	No	500.00
allnnah	DECIMAL (10, 2)	Total loan amount of Al-Innah	Yes	No	500.00
bPulihKenderaan	DECIMAL (10, 2)	Total loan amount of recover vehicle	Yes	No	500.00
roadTaxInsurance	DECIMAL (10, 2)	Total loan amount of road tax insurance	Yes	No	500.00
alQadrulHassan	DECIMAL (10, 2)	Total loan amount of Al-Qadrul Hassan	Yes	No	500.00
specialScheme	VARCHAR (100)	Name of any special scheme availed	Yes	No	KADA Loan Scheme
specialSeasonCarnival	DECIMAL (10, 2)	Special event funds	Yes	No	1500.00
dateUpdated	DATE	Last update date of the financial status	No	No	2024-07-23

transaction	transID (PK)	INT (20)	Unique ID for transaction	No	No	0009
	employeeID (PK)	VARCHAR (8)	Unique ID for member	No	No	0001
	transType	VARCHAR (50)	Type of transaction	No	No	Deposit
	transAmt	DECIMAL (12,2)	Transaction amount	No	No	2000.00
	transDate	DATE	Date of the transaction	No	No	2024-12-10
report	reportID (PK)	INT(20)	Unique ID for report	No	No	0002
	reportTtype	VARCHAR (50)	Type of the report	No	No	Financial
	dateGenerated	DATE	Date the report was generated	No	No	2024-12-10
officeAddress	officeAddress	VARCHAR (100)	Address of member's office	No	No	3A, Jalan Permai
	officePostcode	INT(6)	Postcode of office address	No	No	31400
	officeState	VARCHAR(30)	Office located state	No	No	Perak
homeAddress	homeAddress	VARCHAR (100)	Address of member's house	No	No	36, Jalan Indah Sakti
	homePostcode	INT(6)	Postcode of home address	No	No	31400
	officeState	VARCHAR(30)	Office located state	No	No	Perak

3. Logical Data Model (Logical ERD)

3.1 Logical Database Design (Logical ERD)



3.2 Data Dictionary for the Logical ERD

3.2.1 Entity: employee

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for the user
password	VARCHAR (30)	User's account password

3.2.2 Entity: admin

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for admin
staffName	VARCHAR (80)	Name of the admin

3.2.3 Entity: member

Attribute Name	Type	Description
employeeID (PK)	VARCHAR (8)	Unique ID for member
noPF	INT (4)	Provident fund of the member

memberName (FK)	VARCHAR (80)	Name of the member
ic	CHAR (12)	Identification number of the member
email (FK)	VARCHAR (30)	Member's email address
phoneNumber	VARCHAR (15)	The contact number of the member
sex	CHAR (1)	Gender of the member (M/F)
religion	VARCHAR (30)	The religion of the member
maritalStatus	VARCHAR (15)	Marital status
nation	VARCHAR (30)	Nationality of the member
position	VARCHAR (30)	Working position of the member
monthlySalary	DECIMAL (10,2)	Member's monthly salary
statementID	VARCHAR (20)	Unique ID for financial statement

3.2.4 Entity: memberRegistration

Attribute Name	Type	Description
memberRegistrationID (PK)	VARCHAR (20)	Unique ID for member registration

3.2.5 Entity: memberApplicationDetails

Attribute Name	Type	Description
regisDate	DATE	Date that member apply for loan
regisStatus	VARCHAR (8)	Status of the member registration

3.2.6 Entity: memberApproval

Attribute Name	Type	Description
regisApproveDate	DATE	Date when member registration is approved
regisStatus	VARCHAR (8)	Status of the member registration

3.2.7 Entity: loan

Attribute Name	Type	Description
loanApplicationID (PK)	VARCHAR (20)	Unique ID for loan application
amountRequested	DECIMAL (12,2)	Amount requested for the loan
financingPeriod	INT (2)	Loan repayment period in months
monthlyInstallments	DECIMAL (10,2)	Monthly repayment amount
bankNo	VARCHAR (15)	The bank number account

bankName	VARCHAR (50)	Bank name for loan transfer
financialType	VARCHAR (30)	Type of Financing

3.2.8 Entity: loanApproval

Attribute Name	Type	Description
loanApprovedDate	DATE	Date that admin have approve the loan application
loanStatus	VARCHAR (20)	Status of the loan application

3.2.9 Entity: applicationDetails

Attribute Name	Type	Description
applicationDate	DATE	Date that member apply loan
loanStatus	VARCHAR (20)	Status of the loan application

3.2.10 Entity: guarantor

Attribute Name	Type	Description
icGuarantor (PK)	CHAR (12)	Identification number of the member
guarantorName	VARCHAR (80)	Name of the guarantor
memberIDGuarantor	VARCHAR (8)	Unique ID for member's guarantor
guarantorPFNo	INT (4)	Guarantor's provident fund number
telGuarantor	VARCHAR (15)	Contact number of the member' guarantor

3.2.11 Entity: financialStatus

Attribute Name	Type	Description
accountID (PK)	VARCHAR (8)	Unique ID for member's financial account
statementID	VARCHAR (20)	Unique ID for financial statement
memberSaving	DECIMAL (10,2)	Total savings for the member
alBai	DECIMAL (10, 2)	Total loan amount of Al-Bai
alInnah	DECIMAL (10, 2)	Total loan amount of Al-Innah
bPulihKenderaan	DECIMAL (10, 2)	Total loan amount of recover vehicle
roadTaxInsurance	DECIMAL (10, 2)	Total loan amount of road tax insurance
specialScheme	VARCHAR (100)	Name of any special scheme availed
alQadrulHassan	DECIMAL (10, 2)	Total loan amount of Al-Qadrul Hassan

dateUpdated	DATE	Last update date of the financial status
-------------	------	--

3.2.12 Entity: transaction

Attribute Name	Type	Description
transID (PK)	VARCHAR (20)	Unique ID for transaction
transType	VARCHAR (50)	Type of transaction
transAmt	DECIMAL (12,2)	Transaction amount
transDate	DATE	Date of the transaction

3.2.13 Entity: feesAndContribution

Attribute Name	Type	Description
entryFee	INT (3)	Fee for register as a member
modalShare	DECIMAL (10,2)	The member's share capital or investment in the organization.
feeCapital	DECIMAL (10,2)	Total fee capital of the member
deposit	DECIMAL (10,2)	Total amount deposited by member
contribution	DECIMAL (10,2)	The member's recurring or voluntary financial contribution
fixedDeposit	DECIMAL (10,2)	Amount placed by the member as a fixed deposit with the organization.
others	VARCHAR (50)	Additional fees or contributions

3.2.14 Entity: familyMemberInfo

Attribute Name	Type	Description
icFamilyMember (PK)	CHAR (12)	Identification number of the family member
relationship	VARCHAR (30)	Relationship between family member and member
familyMemberName	VARCHAR (80)	Name of the family member

3.2.15 Entity: homeAddress

Attribute Name	Type	Description
homeAddress	VARCHAR (225)	Address for member's home
homePostcode	VARCHAR (10)	Postcode for member's home

homeState	VARCHAR (20)	State for member's home
-----------	--------------	-------------------------

3.2.16 Entity: officeAddress

Attribute Name	Type	Description
officeAddress	VARCHAR (225)	Address for member's office
officePostcode	VARCHAR (10)	Postcode for member's office
officeState	VARCHAR (20)	State for member's office

3.2.17 Entity: report

Attribute Name	Type	Description
reportType	VARCHAR (10)	Report type, either member, loan, monthly and yearly report
reportFormat	VARCHAR (3)	Report export/download format

3.2.18 Entity: reportDetails

Attribute Name	Type	Description
generateDate	DATE	Date report is generated
reportStatus	VARCHAR (11)	Status of the report, either active or non-active

3.3 Relational Database Schema(s)

EMPLOYEE (employeeID, password)

PK: employeeID

ADMIN (employeeID, staffName)

PK: employeeID

MEMBER(employeeID, noPF, memberName, ic, email, phoneNumber, sex, religion, maritalStatus, nation, position, monthlySalary, statementID)

PK: employeeID

MEMBERREGISTRATION (memberRegistrationID)

PK: memberRegistrationID

MEMBERAPPLICATIONDETAILS (employeeID, memberRegistrationID, regisDate, regisStatus)
PK: employeeID, memberRegistrationID
FK: employeeID references MEMBER(employeeID)
FK: memberRegistrationID references MEMBERREGISTRATION(memberRegistrationID)

MEMBERAPPROVAL(employeeID, memberRegistrationID, regisApproveDate, regisStatus)
PK: employeeID, memberRegistrationID
FK: employeeID references ADMIN(employeeID)
FK: memberRegistrationID references MEMBERREGISTRATION(memberRegistrationID)

LOAN(loanApplicationID, amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, financialType)
PK: loanApplicationID

LOANAPPROVAL(employeeID, loanApplicationID, loanApproveDate, loanStatus)
PK: employeeID, loanApplicationID
FK: employeeID references ADMIN(employeeID)
FK: loanApplicationID references LOAN(loanApplicationID)

APPLICATIONDETAILS (loanApplicationID, employeeID, applicationDate, loanStatus)
PK: loanApplicationID, employeeID
FK: loanApplicationID references LOAN(loanApplicationID)
FK: employeeID references MEMBER(employeeID)

GUARANTOR (icGuarantor, loanApplicationID, guarantorName, memberIDGuarantor, guarantorPFNo, telGuarantor)
PK: icGuarantor, loanApplicationID
FK: loanApplicationID references LOAN (loanApplicationID)

FINANCIALSTATUS (accountID, statementID, memberSaving, alBai, allnnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)
PK: accountID

TRANSACTION(transID, employeeID, accountID, transType, transAmt, transDate)
PK: transID, employeeID, accountID
FK: employeeID references MEMBER(employeeID)
FK: accountID references FINANCIALSTATUS(accountID)

FEESANDCONTRIBUTION (employeeID, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit, others)
PK: employeeID

FK: employeeID references MEMBER(employeeID)

FAMILYMEMBERINFO (icFamilyMember, employeeID, relationship, familyMemberName)

PK: icFamilyMember, employeeID

FK: employeeID references MEMBER(employeeID)

HOMEADDRESS (employeeID, homeAddress, homePostcode, homeState)

PK: employeeID

FK: employeeID references MEMBER(employeeID)

OFFICEADDRESS (employeeID, officeAddress, officePostcode, officeState)

PK: employeeID

FK: employeeID references MEMBER(employeeID)

REPORT(employeeID, loanApplicationID, accountID, reportType, reportFormat)

PK: employeeID, loanApplicationID, accountID

FK: employeeID references MEMBER(employeeID)

FK: loanApplicationID references LOAN(loanApplicationID)

FK: accountID references FINANCIALSTATUS(accountID)

REPORTDETAILS(employeeID, generateDate, reportStatus)

PK: employeeID

FK: employeeID references ADMIN(employeeID)

3.4 Normalization

1. employee - admin

1NF:

employee_admin (employeeID, password, staffName)

PK: employeeID

2NF:

employee (employeeID, password)

PK: employeeID

admin (employeeID, staffName)

PK: employeeID

FK: employeeID references employee(employeeID)

3NF:

employee (employeeID, password)

PK: employeeID
admin (employeeID, staffName)
PK: employeeID
FK: employeeID references employee(employeeID)

2. admin - loanApproval

1NF:
admin_loanApproval(employeeID, staffName, loanApproveDate, loanStatus)
PK: employeeID

2NF:
admin(employeeID, staffName)
PK: employeeID

loanApproval(employeeID, loanApproveDate, loanStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

3NF:
admin(employeeID, staffName)
PK: employeeID

loanApproval(employeeID, loanApproveDate, loanStatus)
PK: employeeID
FK: employeeID references admin(employeeID)

3. loan - loanApproval

1NF:
loanApproval(loanApplicationID, amountRequested, financingPeriod, monthlyInstallments,
bankNo, bankName, loanApproveDate, loanStatus)
PK: loanApplicationID

2NF:
loan(loanApplicationID, amountRequested, financingPeriod, monthlyInstallment)
PK: loanApplicationID
bank(bankNo, bankName)
PK: bankNo
loanApproval (loanApplicationID, bankNo, loanApproveDate, loanStatus)
PK: loanApplicationID, bankNo
FK: loanApplicationID reference loan(loanApplicationID)

FK: bankNo reference bank(bankNo)

3NF:

loan(loanApplicationID, amountRequested, financingPeriod, monthlyInstallment)

PK: loanApplicationID

bank(bankNo, bankName)

PK: bankNo

loanApproval (loanApplicationID, bankNo, loanApproveDate, loanStatus)

PK: loanApplicationID, bankNo

FK: loanApplicationID reference loan(loanApplicationID)

FK: bankNo reference bank(bankNo)

4. loan - guarantorDetails

1NF:

loan_guarantorDetails(loanApplicationID, icGuarantor, amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, guarantorName, memberIDGuarantor, telGuarantor)

PK: loanApplicationID, icGuarantor

2NF:

loan(loanApplicationID, amountRequested, financingPeriod, monthlyInstallment)

PK: loanApplicationID

guarantor(icGuarantor, guarantorName, guarantorPFNo, memberIDGuarantor, telGuarantor)

PK: icGuarantor

loan_guarantorDetails(loanApplicationID, icGuarantor, bankNo)

PK: loanApplicationID, icGuarantor

FK: icGuarantor references guarantor (icGuarantor)

FK: loanApplicationID references loan (loanApplicationID)

3NF:

loan(loanApplicationID, amountRequested, financingPeriod, monthlyInstallment)

PK: loanApplicationID

guarantor(icGuarantor, guarantorName, guarantorPFNo, memberIDGuarantor, telGuarantor)

PK: icGuarantor

loan_guarantorDetails(loanApplicationID, icGuarantor, bankNo)

PK: loanApplicationID, icGuarantor

FK: icGuarantor references guarantor (icGuarantor)

FK: loanApplicationID references loan (loanApplicationID)

5. loan- applicationDetails

1NF:

loan_Application (loanApplicationID, amountRequested, financingPeriod, monthlyInstallments, bankNo, bankName, loanApplicationDate, loanStatus)

PK: loanApplicationID

2NF:

Loan (loanApplicationID, amountRequested, financingPeriod, monthlyInstallments)

PK: loanApplicationID

bank (bankNo, bankName)

PK: bankNo

loan_Application (loanApplicationID, bankNo, loanApplicationDate, loanStatus)

PK: loanApplicationID, bankNo

FK: loanApplicationID reference loan(loanApplicationID)

FK: bankNo reference bank(bankNo)

3NF:

Loan (loanApplicationID, amountRequested, financingPeriod, monthlyInstallments)

PK: loanApplicationID

bank (bankNo, bankName)

PK: bankNo

loan_Application (loanApplicationID, bankNo, loanApplicationDate, loanStatus)

PK: loanApplicationID, bankNo

FK: loanApplicationID reference loan(loanApplicationID)

FK: bankNo reference bank(bankNo)

6. admin - member

1NF:

Admin_Member(employeeID, staffName, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

2NF:

admin(employeeID, staffName)

PK: employeeID

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

admin_member(employeeID)

PK: employeeID

FK: employeeID references admin(employeeID)

FK: employeeID references member(employeeID)

3NF:

admin(employeeID, staffName)

PK: employeeID

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

admin_member(employeeID)

PK: employeeID

FK: employeeID references admin(employeeID)

FK: employeeID references member(employeeID)

7. employee - member

1NF:

employee_member(employeeID, password, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

2NF:

employee(employeeID, password)

PK: employeeID

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

employee_member(employeeID)

PK: employeeID

FK: employeeID references employee(employeeID)

FK: employeeID references member(employeeID)

3NF:

employee(employeeID, password)

PK: employeeID

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

employee_member(employeeID)

PK: employeeID

FK: employeeID references employee(employeeID)

FK: employeeID references member(employeeID)

8. member - applicationDetails

1NF:

memberApplicationDetails(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, applicationDate, loanStatus)

PK: employeeID

2NF:

member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

applicationDetails (employeeID, applicationDate, loanStatus)

PK: employeeID

FK: employeeID references member(employeeID)

3NF:

member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

applicationDetails (employeeID, applicationDate, loanStatus)

PK: employeeID

FK: employeeID references member(employeeID)

9. member - transaction

1NF:

member_transaction (employeeID, transID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, transType, transAmt, transDate)

PK: employeeID, transID

2NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

transaction(transID, transType, transAmt, transDate)

PK: transID

Member_transaction (employeeID, transID)

PK: employeeID, transID

FK: employeeID references member(employeeID)

FK: transID references transaction(transID)

3NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

transaction(transID, transType, transAmt, transDate)

PK: transID

Member_transaction (employeeID, transID)

PK: employeeID, transID

FK: employeeID references member(employeeID)

FK: transID references transaction(transID)

10. member - financialStatus

1NF:

member_financialStatus(employeeID, accountID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: employeeID, accountID

2NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

member_financialStatus(employeeID, accountID)

PK: employeeID, accountID

FK: employee references member(employeeID)

FK: accountID references financialStatus(accountID)

3NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

member_financialStatus(employeeID, accountID)

PK: employeeID, accountID

FK: employee references member(employeeID)

FK: accountID references financialStatus(accountID)

11. transaction - financialStatus

1NF:

transaction_financialStatus (transID, accountID, transType, transAmt, transDate, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: transID, accountID

2NF:

transaction (transID, transType, transAmt, transDate)

PK: transID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

transaction_financialStatus (transID, accountID)

PK: transID, accountID

FK: transID references transaction(transID)

FK: accountID references financialStatus(accountID)

3NF:

transaction (transID, transType, transAmt, transDate)

PK: transID

financialStatus(accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

transaction_financialStatus (transID, accountID)

PK: transID, accountID

FK: transID references transaction(transID)

FK: accountID references financialStatus(accountID)

12. Member - homeAddress

1NF:

Member_homeAddress(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, homeAddress, homePostcode, homeState)

PK: employeeID

2NF:

Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_homeAddress(employeeID, homeAddress, homePostcode, homeState)

PK: employeeID

FK: employeeID references member(employeeID)

3NF:

Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_homeAddress(employeeID, homeAddress, homePostcode, homeState)

PK: employeeID

FK: employeeID references member(employeeID)

13. Member - officeAddress

1NF:

Member_officeAddress(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, officeAddress, officePostcode, officeState)

PK: employeeID

2NF:

Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_officeAddress(employeeID, officeAddress, officePostcode, officeState)

PK: employeeID

FK: employeeID references member(employeeID)

3NF:

Member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_officeAddress(employeeID, officeAddress, officePostcode, officeState)

PK: employeeID

FK: employeeID references member(employeeID)

14. Admin - memberApproval

1NF:

admin_memberApproval(employeeID, staffName, regisApproveDate, regStatus)

PK: employeeID

2NF:

admin_memberApproval(employeeID, staffName, regisApproveDate, regStatus)

PK: employeeID

3NF:

admin_memberApproval(employeeID, staffName, regisApproveDate, regStatus)

PK: employeeID

15. memberApproval - memberRegistration

1NF:

memberApproval_memberRegistration(memberRegistrationID, regisApproveDate, regisStatus)

PK: memberRegistrationID

2NF:

memberApproval_memberRegistration(memberRegistrationID, regisApproveDate, regisStatus)

PK: memberRegistrationID

3NF:

memberApproval_memberRegistration(memberRegistrationID, regisApproveDate, regisStatus)

PK: memberRegistrationID

16. memberRegistration - familyMemberInfo

1NF:

memberRegistration_familyMemberInfo(memberRegistrationID, icFamilyMember, relationship, familyMemberName)

PK: memberRegistrationID, icFamilyMember

2NF:

familyMemberInfo(icFamilyMember, relationship, familyMemberName)

PK: icFamilyMember

memberRegistration_familyMemberInfo(memberRegistrationID, icFamilyMember)

PK: memberRegistrationID, icFamilyMember

FK: icFamilyMember references familyMemberInfo(icFamilyMember)

3NF:

familyMemberInfo(icFamilyMember, relationship, familyMemberName) employeeID
PK: icFamilyMember
memberRegistration_familyMemberInfo(memberRegistrationID, icFamilyMember)
PK: memberRegistrationID, icFamilyMember
FK: icFamilyMember references familyMemberInfo(icFamilyMember)

17. memberRegistration - feesAndContribution

1NF:
memberRegistration_feesAndContribution(memberRegistrationID, entryFee, modalShare,
feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

2NF:
memberRegistration_feesAndContribution(memberRegistrationID, entryFee, modalShare,
feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

3NF:
memberRegistration_feesAndContribution(memberRegistrationID, entryFee, modalShare,
feeCapital, deposit, contribution, fixedDeposit, others)
PK: memberRegistrationID

18. memberRegistration - memberApplicationDetails

1NF:
memberRegistration_memberApplicationDetails(memberRegistrationID, regisDate,
regisStatus)
PK: memberRegistrationID

2NF:
memberRegistration_memberApplicationDetails(memberRegistrationID, regisDate,
regisStatus)
PK: memberRegistrationID

3NF:
memberRegistration_memberApplicationDetails(memberRegistrationID, regisDate,
regisStatus)
PK: memberRegistrationID

19. Member - memberApplicationDetails

1NF:

member_memberApplicationDetails(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, regisDate, regisStatus)

PK: employeeID

2NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_memberApplicationDetails(employeeID, regisDate, regisStatus)

PK: employeeID

FK: employeeID references member(employeeID)

3NF:

member(employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_memberApplicationDetails(employeeID, regisDate, regisStatus)

PK: employeeID

FK: employeeID references member(employeeID)

20. admin_reportDetails

1NF:

admin_reportDetails (employeeID, staffName, generateDate, reportStatus)

PK: employeeID

2NF:

admin (employeeID, staffName)

PK: employeeID

admin_reportDetails (employeeID, generateDate, reportStatus)

PK: employeeID

FK: employeeID references admin(employeeID)

3NF:

admin (employeeID, staffName)

PK: employeeID

admin_reportDetails (employeeID, generateDate, reportStatus)

PK: employeeID

FK: employeeID references admin(employeeID)

21. member_report

1NF:

member_report (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary, reportType, reportFormat)

PK: employeeID

2NF:

member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_report (employeeID, regisDate, regisStatus)

PK: employeeID

FK: employeeID references member(employeeID)

3NF:

member (employeeID, noPF, memberName, ic, email, phoneNumber, phoneHome, sex, religion, maritalStatus, nation, position, monthlySalary)

PK: employeeID

member_report (employeeID, regisDate, regisStatus)

PK: employeeID

FK: employeeID references member(employeeID)

22. financialStatus_report

1NF:

financialStatus_report (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated, reportType, reportFormat)

PK: accountID

2NF:

financialStatus (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

financialStatus_report (accountID, regisDate, regisStatus)

PK: accountID

FK: accountID references financialStatus(accountID)

3NF:

financialStatus (accountID, statementID, memberSaving, alBai, alInnah, bPulihKenderaan, roadTaxInsurance, specialScheme, alQadrulHassan, dateUpdated)

PK: accountID

financialStatus_report (accountID, regisDate, regisStatus)

PK: accountID

FK: accountID references financialStatus(accountID)

23. report_reportDetails

1NF:

report_reportDetails (reportType, reportFormat, generateDate, reportStatus)

PK: reportType, generateDate

2NF:

report_reportDetails (reportType, reportFormat, generateDate, reportStatus)

PK: reportType, generateDate

3NF:

report_reportDetails (reportType, reportFormat, generateDate, reportStatus)

PK: reportType, generateDate

4.0 Interface Design and SQL Implementation

4.1 Interface Design

The interface includes a membership application, loan application, profile page, view financial status, reporting and status page.

4.1.1 Login Interface

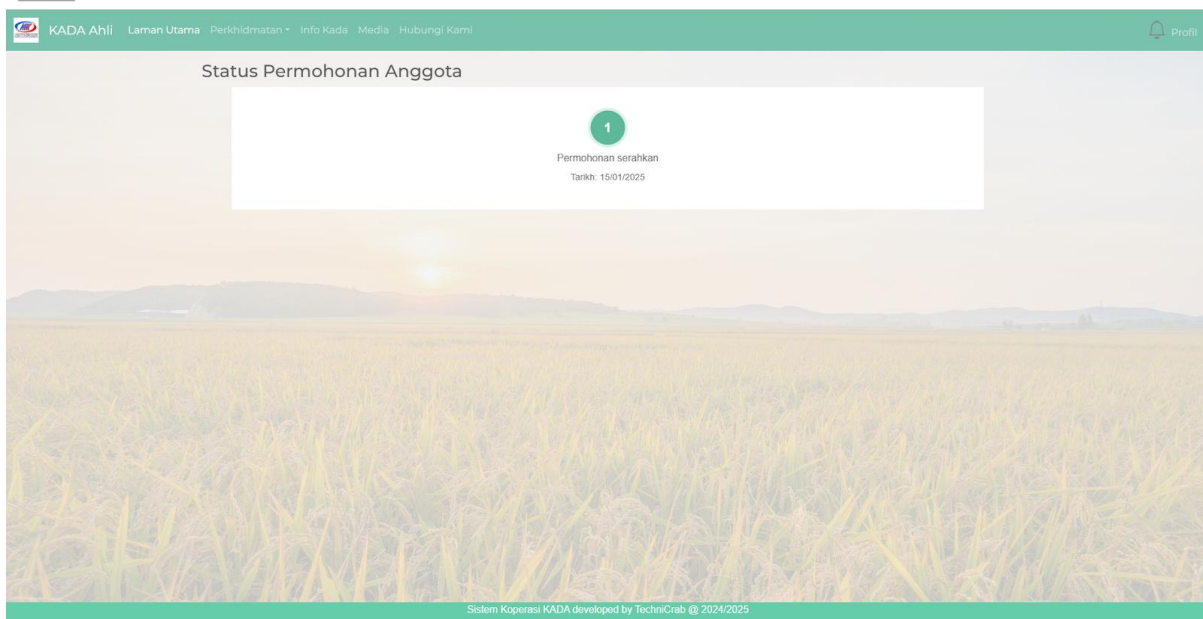
Login interface for both member and admin



If you want to apply membership for koperasi kada and dont have an account yet at our system, you can apply an account



After apply an account and login, you can now apply the membership



4.1.2 Member Interface

Main Page





KADA Ahli

Laman Utama

Perkhidmatan *

Info Kada

Media

Hubungi Kami

Profil

Kembali



NAMA: GUI KAH SIN

NO. KIP: 123445678912

NO. AHLI: 823

NO. PF: 0823

Tuan/Puan,

PENGESEHAN PENYATA KEWANGAN AHLI KOPERASI KAKITANGAN KADA KELANTAN BERAHAD
BAGI TAHUN BERAKHIR 15 Jan 2025

Untuk penentuan Juruaudit, kami dengan ini menyatakan bagi akaun tuan/puan adalah sebagaimana
berikut.

MAKLUMAT SAHAM AHLI:

Modal Saham	RM 0.00
Modal Yuran	RM 0.00
Simpanan Tetap	RM 0.00
Tabung Anggota	RM 0.00

MAKLUMAT PINJAMAN AHLI:

Al-Bai	RM 0.00
Al-Innah	RM 0.00
B/Pulih Kenderaan	RM 0.00
Road Tax & Insuran	RM 0.00

PENGESEHAN BAGI PENYATA KEWANGAN
Saya GUI KAH SIN No. Ahli: 823 mengesahkan bahawa Penyata Kewangan Koperasi Kakitangan KADA
Kelantan Berhad adalah benar.

4.1.3 Admin Interface

Main Page



Member Application



Loan Application



Generate Report

3

Hasil Report

1. Pilih Jenis Laporan

☐ Ahli
 ☐ Pembiayaan

2. Pilih Julat Tarikh

Dalam:

Select Range

Dari:
 Hingga:

3. Pilih Ahli

Jumlah ahli dipilih: 0

Cari ahli...

	No.	Nama	Status	Tarikh Daftar
Sistem Koperasi KADA developed by TechniCrab © 2024/2025				



View Report



4.2 SQL Statements (DDL & DML)

1. Apply membership & View membership application status

1	CREATE TABLE `tb_member` (
2	`employeeID` int(4) NOT NULL,
3	`memberName` varchar(255) NOT NULL,
4	`email` varchar(255) DEFAULT NULL COMMENT 'Email address',
5	`ic` varchar(20) NOT NULL,
6	`maritalStatus` varchar(20) NOT NULL,
7	`sex` varchar(10) DEFAULT NULL,
8	`religion` varchar(50) DEFAULT NULL,

```

9  `nation` varchar(50) DEFAULT NULL,
10 `no_pf` varchar(50) DEFAULT NULL,
11 `position` varchar(100) DEFAULT NULL COMMENT 'Job position',
12 `phoneNumber` varchar(20) DEFAULT NULL COMMENT 'Primary contact number',
13 `phoneHome` varchar(20) DEFAULT NULL COMMENT 'Home contact number',
14 `monthlySalary` decimal(10,2) DEFAULT NULL COMMENT 'Monthly salary amount',
15 `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
16 `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
17 current_timestamp()
18 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
19
20 INSERT INTO `tb_member` (`employeeID`, `memberName`, `email`, `ic`, `maritalStatus`, `sex`,
21 `religion`, `nation`, `no_pf`, `position`, `phoneNumber`, `phoneHome`, `monthlySalary`, `created_at`,
22 `updated_at`) VALUES
23 (322, 'Cheryl Cheong Kah Voon', 'cheryl@gmail.com', '040322040352', 'Bujang', 'Perempuan',
24 'Buddha', 'Cina', '322', '322', '1121192282', '035331234', 2000.00, '2025-01-14 11:18:24', '2025-01-14
25 11:19:10') ;
26
27 ALTER TABLE `tb_member`
28 ADD PRIMARY KEY (`employeeID`);
29
30 CREATE TABLE `tb_memberregistration_familymemberinfo` (
31 `employeeID` int(4) NOT NULL,
32 `relationship` varchar(50) NOT NULL,
33 `name` varchar(100) NOT NULL,
34 `icFamilyMember` varchar(20) NOT NULL
35 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
36
37 INSERT INTO `tb_memberregistration_familymemberinfo` (`employeeID`, `relationship`, `name`,
38 `icFamilyMember`) VALUES
39 (322, 'Anak', 'mini cheryl', '252222-22-2222') ;
40
41 ALTER TABLE `tb_memberregistration_familymemberinfo`
42 ADD PRIMARY KEY (`employeeID`, `icFamilyMember`);
43
44 ALTER TABLE `tb_memberregistration_familymemberinfo`
45 ADD CONSTRAINT `tb_memberregistration_familymemberinfo_ibfk_1` FOREIGN KEY
46 (`employeeID`) REFERENCES `tb_member` (`employeeID`);
47
48
49 CREATE TABLE `tb_memberregistration_feesandcontribution` (
50 `employeeID` int(4) NOT NULL,
51 `entryFee` int(11) NOT NULL DEFAULT 0,
52 `modalShare` int(11) NOT NULL DEFAULT 0,
53 `feeCapital` int(11) NOT NULL DEFAULT 0,
54 `deposit` int(11) NOT NULL DEFAULT 0,
55 `contribution` int(11) NOT NULL DEFAULT 0,
56 `fixedDeposit` int(11) NOT NULL DEFAULT 0,
57 `others` int(11) NOT NULL DEFAULT 0
58 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
59
60
61

```

```

62 INSERT INTO `tb_memberregistration_feesandcontribution` (`employeeID`, `entryFee`,
63 `modalShare`, `feeCapital`, `deposit`, `contribution`, `fixedDeposit`, `others`) VALUES
64 (322, 100, 1000, 0, 0, 600, 1200, 0) ;
65
66 ALTER TABLE `tb_memberregistration_feesandcontribution`
67 ADD PRIMARY KEY (`employeeID`);
68
69 ALTER TABLE `tb_memberregistration_feesandcontribution`
70 ADD CONSTRAINT `fk_member_fees` FOREIGN KEY (`employeeID`) REFERENCES
71 `tb_member` (`employeeID`) ON DELETE CASCADE ON UPDATE CASCADE;
72
73 CREATE TABLE `tb_member_homeaddress` (
74 `employeeID` int(4) NOT NULL,
75 `homeAddress` varchar(255) NOT NULL,
76 `homePostcode` varchar(10) NOT NULL,
77 `homeState` varchar(20) NOT NULL
78 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
79
80 INSERT INTO `tb_member_homeaddress` (`employeeID`, `homeAddress`, `homePostcode`,
81 `homeState`) VALUES
82 (322, 'M19', '82000', 'Melaka') ;
83
84 ALTER TABLE `tb_member_homeaddress`
85 ADD PRIMARY KEY (`employeeID`);
86
87 ALTER TABLE `tb_member_homeaddress`
88 ADD CONSTRAINT `tb_member_homeaddress_ibfk_1` FOREIGN KEY (`employeeID`)
89 REFERENCES `tb_member` (`employeeID`);
90
91 CREATE TABLE `tb_member_officeaddress` (
92 `employeeID` int(4) NOT NULL,
93 `officeAddress` varchar(255) NOT NULL,
94 `officePostcode` varchar(10) NOT NULL,
95 `officeState` varchar(20) NOT NULL
96 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
97
98 INSERT INTO `tb_member_officeaddress` (`employeeID`, `officeAddress`, `officePostcode`,
99 `officeState`) VALUES
100 (322, 'KTDI', '81310', 'Melaka'),
101
102 ALTER TABLE `tb_member_officeaddress`
103 ADD PRIMARY KEY (`employeeID`);
104
105 ALTER TABLE `tb_member_officeaddress`
106 ADD CONSTRAINT `tb_member_officeaddress_ibfk_1` FOREIGN KEY (`employeeID`)
107 REFERENCES `tb_member` (`employeeID`);
108 COMMIT;
109
110 SELECT employeeID FROM tb_member WHERE employeeID = ?;
111
112 INSERT INTO tb_member (
113 employeeID,
114

```

```

115     memberName,
116     email,
117     ic,
118     maritalStatus,
119     sex,
120     religion,
121     nation,
122     no_pf,
123     position,
124     phoneNumber,
125     phoneHome,
126     monthlySalary
127 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
128
129
130 INSERT INTO tb_member_homeaddress (employeeID, homeAddress, homePostcode, homeState)
131 VALUES (?, ?, ?, ?);
132
133 INSERT INTO tb_member_officeaddress (employeeID, officeAddress, officePostcode, officeState)
134 VALUES (?, ?, ?, ?);
135
136
137 INSERT INTO tb_memberregistration_feesandcontribution
138 (employeeID, entryFee, modalShare, feeCapital, deposit, contribution, fixedDeposit)
139 VALUES (?, ?, ?, ?, ?, ?, ?)
140 ON DUPLICATE KEY UPDATE
141 entryFee = VALUES(entryFee),
142 modalShare = VALUES(modalShare),
143 feeCapital = VALUES(feeCapital),
144 deposit = VALUES(deposit),
145 contribution = VALUES(contribution),
146 fixedDeposit = VALUES(fixedDeposit);
147
148
149 DELETE FROM tb_memberregistration_familymemberinfo WHERE employeeID = ?;
150
151 INSERT INTO tb_memberregistration_familymemberinfo
152 (employeeID, relationship, name, icFamilyMember)
153 VALUES (?, ?, ?, ?);
154
155 SELECT
156     CASE
157         WHEN mr.memberRegistrationID IS NOT NULL AND mr.regisStatus = 'Diluluskan' THEN
158         'Diluluskan'
159         WHEN mr.memberRegistrationID IS NOT NULL AND mr.regisStatus = 'Ditolak' THEN
160         'Ditolak'
161         WHEN mr.memberRegistrationID IS NOT NULL THEN 'Belum Selesai'
162         ELSE 'Tiada Rekod'
163     END as regisStatus,
164     CASE
165         WHEN mr.regisDate IS NOT NULL THEN DATE(mr.regisDate)
166         WHEN m.created_at IS NOT NULL THEN DATE(m.created_at)
167         ELSE CURRENT_DATE()

```

```

168     END as regisDate
169 FROM tb_member m
170 LEFT JOIN tb_memberregistration_memberapplicationdetails mr
171 ON m.employeeID = mr.memberRegistrationID
172 WHERE m.employeeID = ?;
173

```

2. Apply for loan & View loan application status

```

1  CREATE TABLE `tb_bank` (
2  `bankID` int(11) NOT NULL,
3  `loanApplicationID` int(11) NOT NULL,
4  `employeeID` int(4) NOT NULL,
5  `bankName` varchar(100) NOT NULL,
6  `accountNo` varchar(50) NOT NULL
7  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
8
9  INSERT INTO `tb_bank` (`bankID`, `loanApplicationID`, `employeeID`, `bankName`,
10 `accountNo`) VALUES
11 (28, 36, 322, 'Affin Bank', '1234'),
12
13 ALTER TABLE `tb_bank`
14   ADD PRIMARY KEY (`bankID`),
15   ADD KEY `loanApplicationID` (`loanApplicationID`);
16
17 ALTER TABLE `tb_bank`
18   MODIFY `bankID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=35;
19
20 ALTER TABLE `tb_bank`
21   ADD CONSTRAINT `tb_bank_ibfk_1` FOREIGN KEY (`loanApplicationID`) REFERENCES
22 `tb_loanapplication` (`loanApplicationID`);
23
24
25 CREATE TABLE `tb_guarantor` (
26 `guarantorID` int(11) NOT NULL,
27 `loanApplicationID` int(11) DEFAULT NULL,
28 `employeeID` int(11) DEFAULT NULL,
29 `guarantorName` varchar(255) NOT NULL,
30 `guarantorIC` varchar(20) NOT NULL,
31 `guarantorPhone` varchar(20) NOT NULL,
32 `guarantorPFNo` varchar(20) NOT NULL,
33 `guarantorMemberNo` varchar(20) NOT NULL
34 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
35
36
37 INSERT INTO `tb_guarantor` (`guarantorID`, `loanApplicationID`, `employeeID`,
38 `guarantorName`, `guarantorIC`, `guarantorPhone`, `guarantorPFNo`, `guarantorMemberNo`)
39 VALUES

```



```

40 (5, 36, 322, 'CHERYL', '111111111111', '1111111111', '111', '1111'),
41
42 ALTER TABLE `tb_guarantor`
43   ADD PRIMARY KEY (`guarantorID`);
44
45 ALTER TABLE `tb_guarantor`
46   MODIFY `guarantorID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=17;
47
48
49
50 CREATE TABLE `tb_loan` (
51   `loanType` varchar(30) NOT NULL,
52   `loanID` int(11) NOT NULL,
53   `loanApplicationID` int(11) NOT NULL,
54   `employeeID` int(4) NOT NULL,
55   `amountRequested` decimal(10,2) NOT NULL COMMENT 'amount that want to pinjam',
56   `financingPeriod` int(11) NOT NULL,
57   `monthlyInstallments` decimal(10,2) NOT NULL,
58   `employerName` varchar(100) NOT NULL,
59   `employerIC` varchar(14) NOT NULL,
60   `basicSalary` decimal(10,2) NOT NULL,
61   `netSalary` decimal(10,2) NOT NULL,
62   `basicSalaryFile` varchar(255) NOT NULL,
63   `netSalaryFile` varchar(255) NOT NULL,
64   `balance` decimal(10,2) DEFAULT NULL,
65   `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
66   `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE
67   current_timestamp()
68 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
69
70 INSERT INTO `tb_loan` (`loanType`, `loanID`, `loanApplicationID`, `employeeID`,
71 `amountRequested`, `financingPeriod`, `monthlyInstallments`, `employerName`, `employerIC`,
72 `basicSalary`, `netSalary`, `basicSalaryFile`, `netSalaryFile`, `balance`, `created_at`, `updated_at`)
73 VALUES
74 ('AL-BAI', 20, 36, 322, 3000.00, 12, 250.00, 'NAMA MAJIKAN', '222222222222', 2000.00,
75 1500.00, 'uploads/6786a2674a8b4_MAFEST \'25 UNIT HADIAH (2).pdf',
76 'uploads/6786a2674aa3b_OPERA \'25 UNIT HADIAH (2).pdf', NULL, '2025-01-18 16:46:10',
77 '2025-01-18 16:47:01'),
78
79 ALTER TABLE `tb_loan`
80   ADD PRIMARY KEY (`loanID`),
81   ADD KEY `loanApplicationID` (`loanApplicationID`);
82
83 ALTER TABLE `tb_loan`
84   MODIFY `loanID` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=26;
85
86 ALTER TABLE `tb_loan`
87   ADD CONSTRAINT `tb_loan_ibfk_1` FOREIGN KEY (`loanApplicationID`) REFERENCES
88 `tb_loanapplication` (`loanApplicationID`);
89
90 ALTER TABLE `tb_loan`
100   DROP `basicSalaryFile`;
101

```

```

102
103
104 CREATE TABLE `tb_loanapplication` (
105   `loanApplicationID` int(11) NOT NULL,
106   `employeeID` int(4) NOT NULL,
107   `loanApplicationDate` date NOT NULL,
108   `loanStatus` varchar(20) DEFAULT 'Pending',
109   `amountRequested` decimal(10,2) NOT NULL,
110   `financingPeriod` int(11) NOT NULL,
111   `monthlyInstallments` decimal(10,2) NOT NULL
112 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
113
114
115 INSERT INTO `tb_loanapplication` (`loanApplicationID`, `employeeID`, `loanApplicationDate`,
116 `loanStatus`, `amountRequested`, `financingPeriod`, `monthlyInstallments`) VALUES
117 (36, 322, '2025-01-15', 'Diluluskan', 3000.00, 12, 250.00),
118
119 ALTER TABLE `tb_loanapplication`
120   ADD PRIMARY KEY (`loanApplicationID`);
121
122 ALTER TABLE `tb_loanapplication`
123   MODIFY `loanApplicationID` int(11) NOT NULL AUTO_INCREMENT,
124   AUTO_INCREMENT=43;
125
126
127 -- Check duplicate loan applications
128 SELECT COUNT(*)
129 FROM tb_loanapplication
130 WHERE employeeID = ?
131 AND loanStatus IN ('Dalam Proses', 'Diluluskan')
132 AND loanType = ?
133
134
135 -- Check registration status
136 SELECT COALESCE(mr.regisStatus, 'Belum Selesai') as regisStatus
137 FROM tb_member m
138 LEFT JOIN tb_memberregistration_memberapplicationdetails mr
139 ON m.employeeID = mr.memberRegistrationID
140 WHERE m.employeeID = ?
141
142
143 -- Get member details
144 SELECT m.*, h.homeAddress, h.homePostcode, h.homeState,
145        o.officeAddress, o.officePostcode, o.officeState
146 FROM tb_member m
147 LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
148 LEFT JOIN tb_member_officeaddress o ON m.employeeID = o.employeeID
149 WHERE m.employeeID = ?
150
151
152 -- Get latest loan application
153 SELECT loanApplicationID FROM tb_loanapplication
154 WHERE employeeID = ?

```

```

155 ORDER BY created_at DESC LIMIT 1
156
157
158 -- Update loan status
159 UPDATE tb_loanapplication
160 SET loanStatus = 'Dalam Proses'
161 WHERE loanApplicationID = ?
162
163
164 -- Update loan balance
165 UPDATE tb_loan
166 SET balance = amountRequested
167 WHERE loanApplicationID = ?
168
169
170 SELECT memberName
171 FROM tb_member
172 WHERE employeeID = ?
173 SELECT
174     la.loanApplicationID,
175     la.loanApplicationDate,
176     la.loanStatus,
177     la.amountRequested,
178     l.loanType,
179     l.financingPeriod,
180     l.monthlyInstallments,
181     l.balance
182 FROM tb_loanapplication la
183 LEFT JOIN tb_loan l ON l.loanApplicationID = la.loanApplicationID
184 WHERE la.employeeID = ?
185 ORDER BY la.loanApplicationID DESC
186
187
188

```

3. Approve member registration

```

1 SELECT
2     m.employeeID as memberRegistrationID,
3     m.memberName,
4     m.ic,
5     m.created_at as regisDate,
6     COALESCE(
7         (SELECT regisStatus
8          FROM tb_memberregistration_memberapplicationdetails
9          WHERE memberRegistrationID = m.employeeID
10         ORDER BY regisDate DESC
11         LIMIT 1),

```

```

12      'Belum Selesai'
13    ) as regisStatus
14  FROM
15    tb_member m
16  LEFT JOIN tb_memberregistration_memberapplicationdetails md
17    ON m.employeeID = md.memberRegistrationID
18  GROUP BY
19    m.employeeID, m.memberName, m.ic, m.created_at;
20
21
22  SELECT memberRegistrationID
23  FROM tb_memberregistration_memberapplicationdetails
24  WHERE memberRegistrationID = ?
25
26
27  UPDATE tb_memberregistration_memberapplicationdetails
28  SET regisStatus = ?, regisDate = ?
29  WHERE memberRegistrationID = ?
30
31
32  INSERT INTO tb_memberregistration_memberapplicationdetails
33  (memberRegistrationID, regisDate, regisStatus)
34  VALUES (?, ?, ?)
35
36
37  SELECT
38    m.*,
39    COALESCE(mha.homeAddress, '') as homeAddress,
40    COALESCE(mha.homePostcode, '') as homePostcode,
41    COALESCE(mha.homeState, '') as homeState,
42    COALESCE(moa.officeAddress, '') as officeAddress,
43    COALESCE(moa.officePostcode, '') as officePostcode,
44    COALESCE(moa.officeState, '') as officeState,
45    COALESCE(mfc.entryFee, 0) as entryFee,
46    COALESCE(mfc.modalShare, 0) as modalShare,
47    COALESCE(mfc.feeCapital, 0) as feeCapital,
48    COALESCE(mfc.deposit, 0) as deposit,
49    COALESCE(mfc.contribution, 0) as contribution,
50    COALESCE(mfc.fixedDeposit, 0) as fixedDeposit,
51    COALESCE(mfc.others, 0) as others
52  FROM tb_member m
53  LEFT JOIN tb_member_homeaddress mha ON m.employeeID = mha.employeeID
54  LEFT JOIN tb_member_officeaddress moa ON m.employeeID = moa.employeeID
55  LEFT JOIN tb_memberregistration_feesandcontribution mfc ON m.employeeID = mfc.employeeID
56  WHERE m.employeeID = ?
57
58
59  SELECT *
60  FROM tb_memberregistration_familymemberinfo
61  WHERE employeeID = ?
62

```

4. Approve loan application

```
1 SELECT
2   la.loanApplicationID,
3   m.memberName,
4   m.ic,
5   la.loanApplicationDate as applicationDate,
6   COALESCE(la.loanStatus, 'Belum Selesai') as loanStatus
7 FROM
8   tb_loanapplication la
9 JOIN
10  tb_member m ON la.employeeID = m.employeeID
11 ORDER BY
12   la.loanApplicationDate DESC;
13
14
15 UPDATE tb_loanapplication
16 SET loanStatus = ?,
17    loanApplicationDate = CASE
18     WHEN loanStatus != ? THEN CURRENT_DATE
19     ELSE loanApplicationDate
20 END
21 WHERE loanApplicationID = ?
22
23
24 SELECT
25   la.loanApplicationID,
26   la.amountRequested,
27   la.financingPeriod,
28   la.monthlyInstallments,
29   l.loanType,
30   l.employerName,
31   l.employerIC,
32   l.basicSalary,
33   l.netSalary,
34   m.memberName,
35   m.ic,
36   m.maritalStatus,
37   m.sex,
38   m.religion,
39   m.nation,
40   m.no_pf,
41   m.phoneNumber,
42   mha.homeAddress,
43   mha.homePostcode,
44   mha.homeState,
45   moa.officeAddress,
46   moa.officePostcode,
47   moa.officeState,
48   b.bankName,
```

49	b.accountNo,
50	l.basicSalaryFile,
51	l.netSalaryFile
52	FROM tb_loanapplication la
53	LEFT JOIN tb_loan l ON l.loanApplicationID = la.loanApplicationID
54	LEFT JOIN tb_member m ON la.employeeID = m.employeeID
55	LEFT JOIN tb_member_homeaddress mha ON m.employeeID = mha.employeeID
56	LEFT JOIN tb_member_officeaddress moa ON m.employeeID = moa.employeeID
57	LEFT JOIN tb_bank b ON b.loanApplicationID = la.loanApplicationID
58	WHERE la.loanApplicationID = ?
59	
60	
61	SELECT basicSalary, netSalary FROM tb_loan WHERE loanApplicationID = ?
62	
63	
64	SELECT * FROM tb_guarantor WHERE loanApplicationID = ?
65	

5. Manage Pprofile

1	//Profile which the member already applies a loan
2	SELECT m.*,
3	h.homeAddress, h.homePostcode, h.homeState,
4	o.officeAddress, o.officePostcode, o.officeState
5	FROM tb_member m
6	LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
7	LEFT JOIN tb_member_officeaddress o ON m.employeeID = o.employeeID
8	WHERE m.employeeID = ?
9	
10	
11	//Profile which the user hasn't applied for membership and member hasn't apply for
12	a loan
13	if (\$isMember) {
14	\$sql = "SELECT m.*,
15	h.homeAddress, h.homePostcode, h.homeState,
16	o.officeAddress, o.officePostcode, o.officeState
17	FROM tb_member m
18	LEFT JOIN tb_member_homeaddress h ON m.employeeID = h.employeeID
19	LEFT JOIN tb_member_officeaddress o ON m.employeeID =
20	o.employeeID
21	WHERE m.employeeID = ?";
22	} else {
23	\$sql = "SELECT e.*
24	FROM tb_employee e

```

25         WHERE e.employeeID = ?";
26     }
27
28 //Update profile page
29 UPDATE tb_member SET
30     memberName = ?,
31     email = ?,
32     ic = ?,
33     maritalStatus = ?,
34     sex = ?,
35     religion = ?,
36     nation = ?,
37     phoneNumber = ?
38     WHERE employeeID = ?
39
40 //update home address
41 UPDATE tb_member_homeaddress SET
42     homeAddress = ?,
43     homePostcode = ?,
44     homeState = ?
45     WHERE employeeID = ?
46
47 //update office address
48 UPDATE tb_member_officeaddress SET
49     officeAddress = ?,
50     officePostcode = ?,
51     officeState = ?
52     WHERE employeeID = ?
53
54

```

6. View financial status

1	CREATE TABLE `tb_transaction` (
2	`employeeID` int(4) NOT NULL,
3	`transType` varchar(10) NOT NULL,
4	`transAmt` int(6) NOT NULL,
5	`transDate` date NOT NULL,
6	`transID` int(10) NOT NULL
7	) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
8	COLLATE=utf8mb4_general_ci;
9	
10	INSERT INTO `tb_transaction` (`employeeID`, `transType`, `transAmt`,
11	`transDate`, `transID`) VALUES
12	(4567, 'Simpanan-M', 300, '2025-01-19', 53),
13	(4567, 'Simpanan-Y', 40, '2025-01-19', 54),
14	(4567, 'Simpanan-S', 40, '2025-01-19', 55),
15	(4567, 'Simpanan-T', 40, '2025-01-19', 56),
16	(4567, 'Bayaran Ba', 750, '2025-01-19', 57),
17	(322, 'Simpanan-M', 1000, '2025-01-03', 58),
18	(322, 'Simpanan-S', 1200, '2025-01-03', 59),
19	(322, 'Simpanan-T', 600, '2025-01-03', 60),
20	(322, 'Simpanan-M', 1000, '2025-01-03', 61),
21	(322, 'Simpanan-S', 1200, '2025-01-03', 62),
22	(322, 'Simpanan-T', 600, '2025-01-03', 63),
23	(322, 'Bayaran Ba', 250, '2025-01-11', 64),
24	(322, 'Bayaran Ba', 250, '2025-01-11', 65),
25	(322, 'Bayaran Ba', 250, '2025-01-10', 66),
26	(9876, 'Entry Fee', 20, '2025-01-19', 80),
27	(9876, 'Deposit', 20, '2025-01-19', 81),
28	(9876, 'Simpanan-M', 300, '2025-01-19', 82),
29	(9876, 'Simpanan-Y', 20, '2025-01-19', 83),
30	(9876, 'Simpanan-S', 20, '2025-01-19', 84),
31	(9876, 'Simpanan-T', 20, '2025-01-19', 85),
32	(9876, 'Entry Fee', 20, '2025-02-19', 86),
33	(9876, 'Deposit', 20, '2025-02-19', 87),
34	(9876, 'Simpanan-M', 300, '2025-02-19', 88),
35	(9876, 'Simpanan-Y', 20, '2025-02-19', 89),
36	(9876, 'Simpanan-S', 20, '2025-02-19', 90),
37	(9876, 'Simpanan-T', 20, '2025-02-19', 91),
38	(9876, 'Entry Fee', 20, '2025-03-19', 92),
39	(9876, 'Deposit', 20, '2025-03-19', 93),
40	(9876, 'Simpanan-M', 300, '2025-03-19', 94),
41	(9876, 'Simpanan-Y', 20, '2025-03-19', 95),
42	(9876, 'Simpanan-S', 20, '2025-03-19', 96),
43	(9876, 'Simpanan-T', 20, '2025-03-19', 97),


```

44 (9876, 'Bayaran Ba', 250, '2025-01-11', 98),
45 (9876, 'Bayaran Ba', 250, '2025-02-11', 99),
46 (9876, 'Bayaran Ba', 250, '2025-03-11', 101);
47
48 CREATE TABLE `tb_financialstatus` (
49   `employeeID` int(4) NOT NULL,
50   `modalShare` int(11) NOT NULL,
51   `feeCapital` int(11) NOT NULL,
52   `contribution` int(11) NOT NULL,
53   `fixedDeposit` int(11) NOT NULL,
54   `dateUpdated` int(10) NOT NULL
55 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
56 COLLATE=utf8mb4_general_ci;
57
58 INSERT INTO `tb_financialstatus` (`employeeID`, `modalShare`, `feeCapital`,
59 `contribution`, `fixedDeposit`, `dateUpdated`) VALUES
60 (322, 0, 0, 0, 0, 2147483647),
61 (1234, 0, 0, 0, 0, 2147483647),
62 (5522, 0, 0, 0, 0, 2147483647),
63 (9876, 900, 60, 60, 60, 1737350531);
64
65
66
67 //fetch financial data
68 SELECT * FROM tb_memberregistration_feesandcontribution
69     WHERE employeeID = ?
70
71 //get the transaction amount
72 SELECT transType, transAmt
73     FROM tb_transaction
74     WHERE employeeID = ?
75
76 //get fee distribution
77 SELECT
78     entryFee,
79     deposit,
80     modalShare,
81     feeCapital,
82     contribution,
83     fixedDeposit
84 FROM tb_memberregistration_feesandcontribution
85 WHERE employeeID = ?
86
87 //get all transaction

```

```

88 SELECT transAmt, transDate
89     FROM tb_transaction
90     WHERE employeeID = ?
91     ORDER BY transDate ASC
92
93
94 //fatech bank no
95 SELECT accountNo FROM tb_bank WHERE employeeID = ? ORDER BY
96 bankID DESC LIMIT 1
97
98 //update balance loan
99 UPDATE tb_loan l
100 JOIN tb_loanapplication la ON l.employeeID = la.employeeID
101 SET l.balance = CASE
102     WHEN l.balance IS NULL THEN la.amountRequested
103     ELSE l.balance
104 END
105 WHERE l.employeeID = ?
106 AND la.loanStatus = 'Diluluskan'
107
108
109 //fetch amount already paid
110 SELECT SUM(transAmt) as total_paid
111 FROM tb_transaction
112 WHERE employeeID = ?
113 AND transType = 'Bayaran Ba'
114
115 //fetch loan data
116 SELECT
117     l.loanType,
118     l.balance,
119     la.amountRequested,
120     la.monthlyInstallments
121 FROM tb_loan l
122 JOIN tb_loanapplication la ON l.employeeID = la.employeeID
123 WHERE l.employeeID = ?
124 AND la.loanStatus = 'Diluluskan'
125
126 //update balance in tb_loan
127 UPDATE tb_loan
128     SET balance = ?
129     WHERE employeeID = ?
130     AND loanType = ?
131

```

```

132
133 //fetch latest amount
134 SELECT
135     modalShare,
136     feeCapital,
137     fixedDeposit,
138     contribution
139 FROM tb_financialstatus
140 WHERE employeeID = ?
141
142 //update tb_financialStatus
143 INSERT INTO tb_financialstatus
144     (employeeID, modalShare, feeCapital, contribution, fixedDeposit,
145     dateUpdated)
146     VALUES (?, ?, ?, ?, ?, UNIX_TIMESTAMP())
147     ON DUPLICATE KEY UPDATE
148     modalShare = VALUES(modalShare),
149     feeCapital = VALUES(feeCapital),
150     contribution = VALUES(contribution),
151     fixedDeposit = VALUES(fixedDeposit),
152     dateUpdated = UNIX_TIMESTAMP()
153
154
155 //fetch amount for each loan type
156 SELECT
157     l.loanType,
158     l.balance
159 FROM tb_loan l
160 JOIN tb_loanapplication la ON l.employeeID = la.employeeID
161 WHERE l.employeeID = ?
162 AND la.loanStatus = 'Diluluskan'
163
164 //view financial statement
165 //fetch member detail
1661 SELECT m.*, f.*
167     FROM tb_member m
168     LEFT JOIN tb_memberregistration_feesandcontribution f ON
169     m.employeeID = f.employeeID
170     WHERE m.employeeID = ?
171
172 //fetch loan data
173 SELECT
174     SUM(CASE WHEN l.loanType = 'AL-BAI' THEN l.amountRequested ELSE 0
175     END) as alBai,

```

```

176 SUM(CASE WHEN l.loanType = 'AL-INAH' THEN l.amountRequested ELSE 0
177 END) as alnnah,
178 SUM(CASE WHEN l.loanType = 'B/PULIH KENDERAAN' THEN
179 l.amountRequested ELSE 0 END) as bPulihKendaraan,
180 SUM(CASE WHEN l.loanType = 'ROAD TAX & INSURAN' THEN
181 l.amountRequested ELSE 0 END) as roadTaxInsurance
182 FROM tb_loan l
183 JOIN tb_loanapplication la ON l.loanApplicationID = la.loanApplicationID
184 WHERE l.employeeID = ? AND la.loanStatus = 'Diluluskan'
185
1861 //view monthly statement
87 //fetch month
188 SELECT DISTINCT YEAR(transDate) as year, MONTH(transDate) as month
189 FROM tb_transaction
190 WHERE employeeID = ?
191 ORDER BY year DESC, month DESC
192
193 //fetch member detail
194 SELECT m.*, f.*
195 FROM tb_member m
196 LEFT JOIN tb_memberregistration_feesandcontribution f ON
197 m.employeeID = f.employeeID
198 WHERE m.employeeID = ?
199
200 //fetch first time payment
201 SELECT MIN(transDate) as first_payment
202 FROM tb_transaction
203 WHERE employeeID = ?
204
205 //fetch transaction detail
206 SELECT t.transDate, t.transType, t.transAmt
207 FROM tb_transaction t
208 WHERE t.employeeID = ?
209 AND MONTH(t.transDate) = ?
210 AND YEAR(t.transDate) = ?
211 if (!$is_first_month) {
212 $sql .= " AND t.transType NOT IN ('Entry Fee', 'Deposit')";
213 }
214
215 $sql .= " ORDER BY t.transDate DESC";
216
217
218 //fetch loan detail
219 SELECT

```

```

220     l.loanType,
221     la.amountRequested,
222     la.monthlyInstallments
223 FROM tb_loan l
224 JOIN tb_loanapplication la ON l.employeeID = la.employeeID
225 WHERE l.employeeID = ?
226 AND la.loanStatus = 'Diluluskan'
227
228 //fetch the transaction for selected month
229 SELECT SUM(transAmt) as total_paid
230         FROM tb_transaction
231         WHERE employeeID = ?
232         AND transType = 'Bayaran Ba'
233         AND (
234             YEAR(transDate) < ?
235             OR (YEAR(transDate) = ? AND MONTH(transDate) <= ?)
236         )
237
238 //fetch the amount for selected month
239 SELECT
240     SUM(CASE WHEN transType = 'Simpanan-M' THEN transAmt ELSE 0 END)
241 as modalShare,
242     SUM(CASE WHEN transType = 'Simpanan-Y' THEN transAmt ELSE 0 END)
243 as feeCapital,
244     SUM(CASE WHEN transType = 'Simpanan-T' THEN transAmt ELSE 0 END)
245 as contribution,
246     SUM(CASE WHEN transType = 'Simpanan-S' THEN transAmt ELSE 0 END)
247 as fixedDeposit
248 FROM tb_transaction
249 WHERE employeeID = ?
250 AND (
251     YEAR(transDate) < ?
252     OR (YEAR(transDate) = ? AND MONTH(transDate) <= ?)
253 )
254 AND transType IN ('Simpanan-M', 'Simpanan-Y', 'Simpanan-T', 'Simpanan-S')
255
256 //track the transaction history
257 //fetch the first time payment
258 SELECT MIN(transDate) as first_payment
259         FROM tb_transaction
260         WHERE employeeID = ?
261
262 //fetch the transaction detail
263 SELECT t.transDate, t.transType, t.transAmt

```

264	FROM tb_transaction t
265	WHERE t.employeeID = ?
266	AND MONTH(t.transDate) = ?
267	AND YEAR(t.transDate) = ?
268	if (!\$is_first_month) {
269	\$sql .= " AND t.transType NOT IN ('Entry Fee', 'Deposit')";
270	}
271	
272	\$sql .= " ORDER BY t.transDate DESC";
273	
274	
275	//fetch the date of first time payment
276	SELECT MIN(transDate) as first_payment
277	FROM tb_transaction
278	WHERE employeeID = ?
2792	
80	

5.0 Summary

The Koperasi KADA Online System, a revolutionary project that aims to modernise the cooperative's operations by substituting the current manual processes with an automated, secure, and user-friendly digital platform, is described in this report along with its design and implementation. The suggested system significantly improves operational efficiency and user satisfaction while addressing the shortcomings of the current system, including its excessive reliance on human resources, error-proneness, and insufficient data security.

The report provides with an overview of Koperasi Kakitangan KADA Kelantan Berhad, emphasising the organization's goal of promoting its members' social and economic well-being. It then gives a thorough rundown of the difficulties with the manual system, such as laborious procedures, a lack of services, and dangers related to physical data storage. Online registration, real-time application tracking, automated loan processing, and secure financial record management are among the solutions offered by the proposed Koperasi KADA Online System.

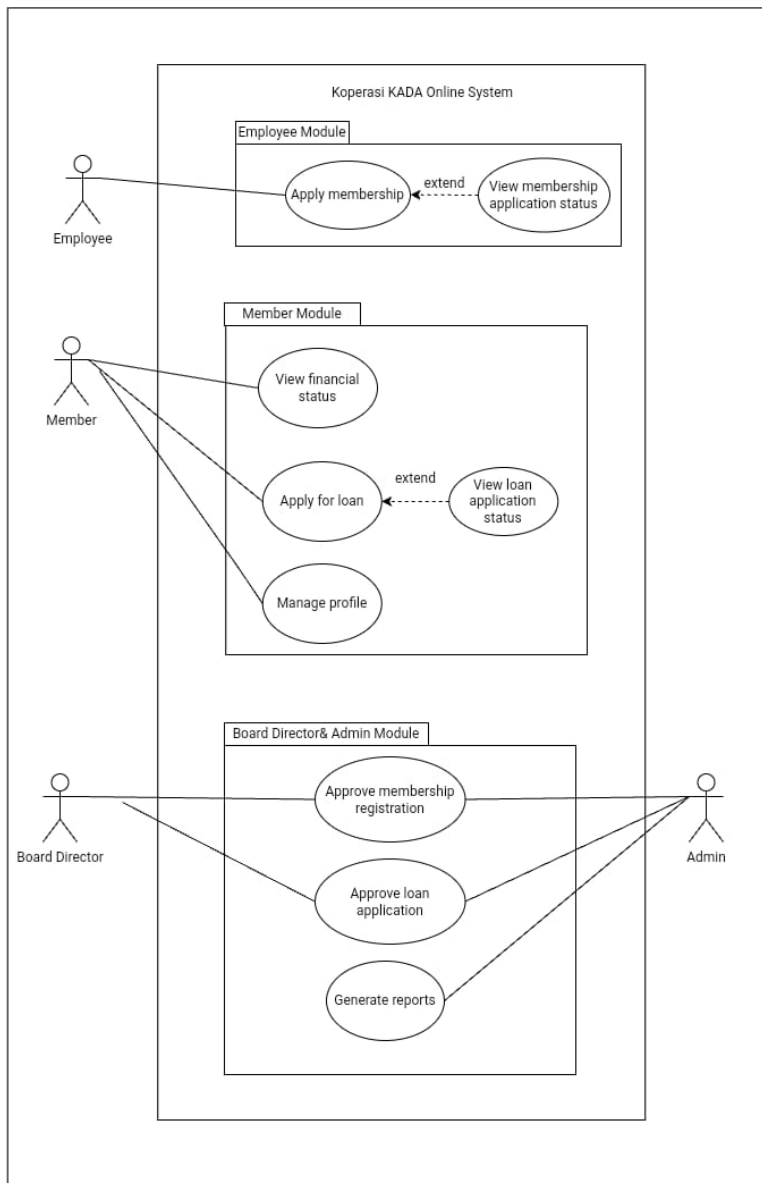
The database design section includes a Global Conceptual Data Model , an Entity-Relationship Diagram (ERD), and a data dictionary to ensure data consistency and integration. The Logical Data Model enhances this design by normalising data structures up to the third normal form (3NF), which improves database performance and reduces redundancy. The Interface Design and SQL Implementation sections demonstrate a user-centric system with tailored interfaces for members and administrators, as well as robust SQL scripts for efficient and secure data management.

By automating key workflows and implementing advanced security measures such as Two-Factor Authentication (2FA), the system reduces human error, workload, and protects sensitive member data. The platform improves the overall user experience by giving members easy access to services and real-time updates, while staff can focus on strategic tasks.

In conclusion, the Koperasi KADA Online System is a significant step forward in the cooperative's digital transformation. This scalable and secure platform aligns with KADA's mission, allowing the organisation to improve services, increase member satisfaction, and achieve operational excellence.

6.0 Appendices

6.1 Use Case Diagram for the system



6.2 Activity Diagrams for every use case identified

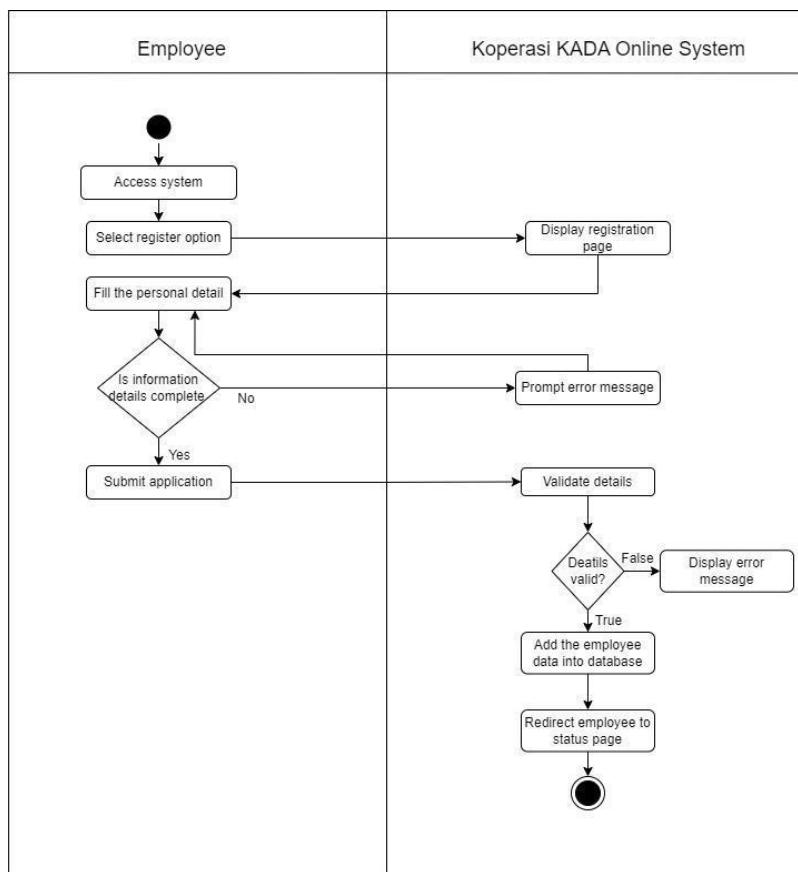


Figure 6.2.1 Activity Diagram for Apply Membership

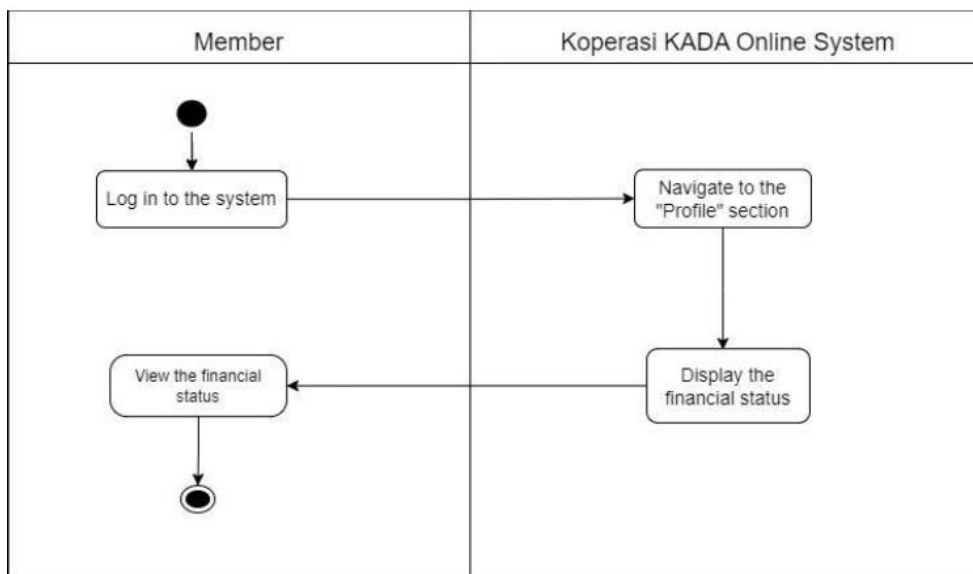


Figure 6.2.2 Activity Diagram for View Financial Status

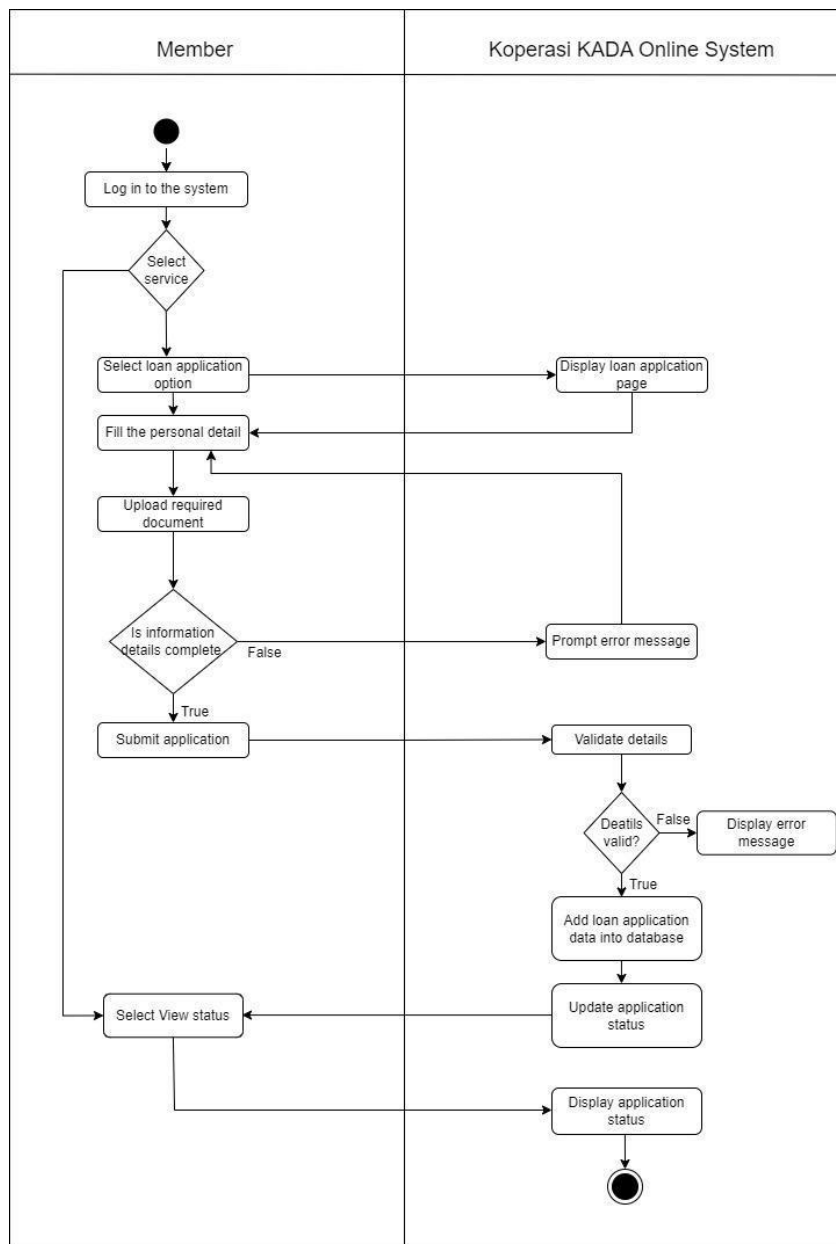


Figure 6.2.3 Activity Diagram for Apply for Loan

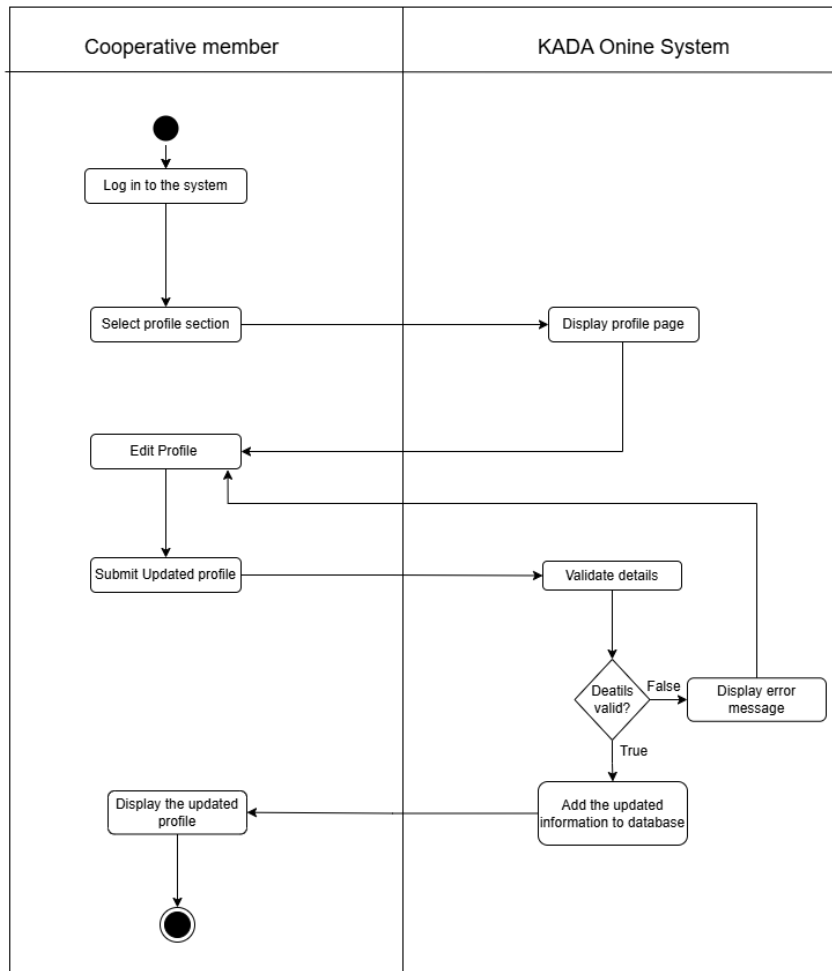


Figure 6.2.4 Activity Diagram for Manage Profile

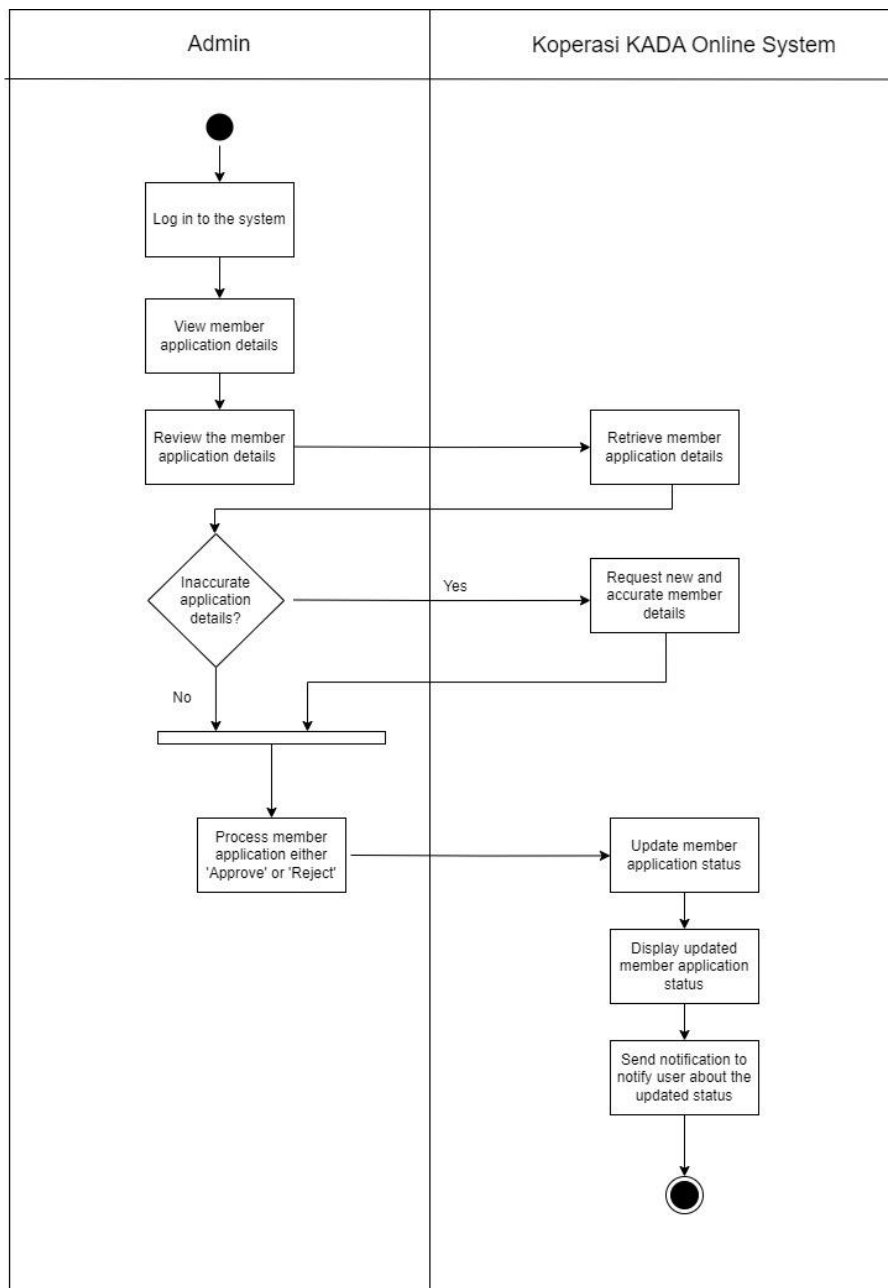


Figure 6.2.5 Activity Diagram for Approve Membership Application

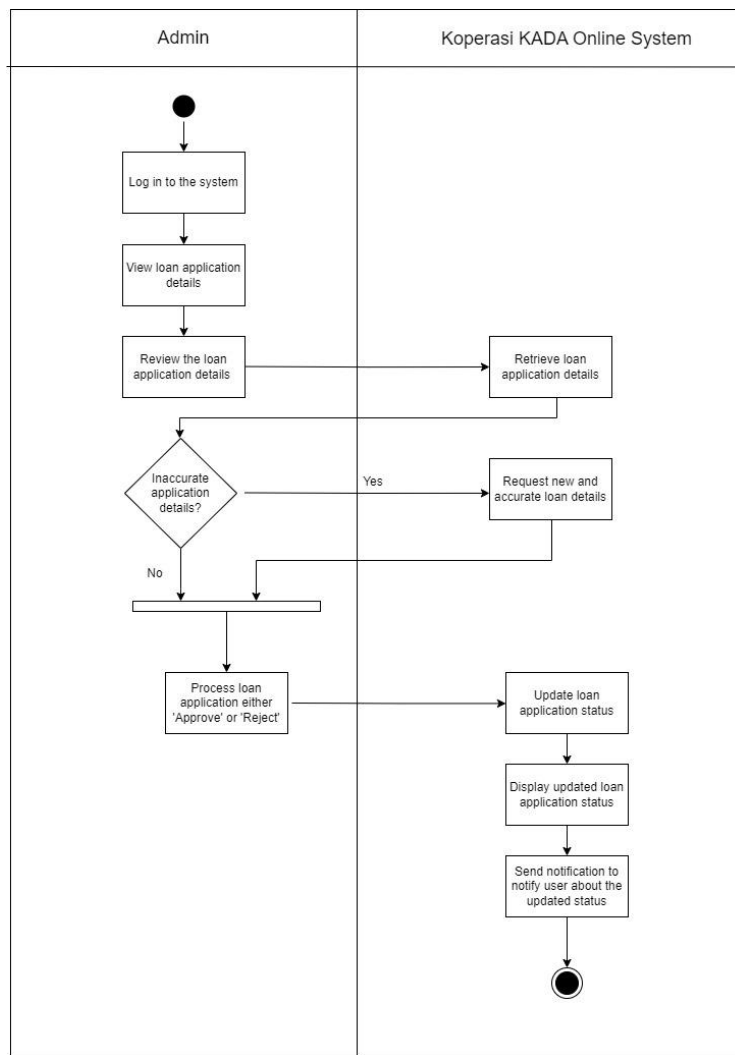


Figure 6.2.6 Activity Diagram for Approve Loan Application

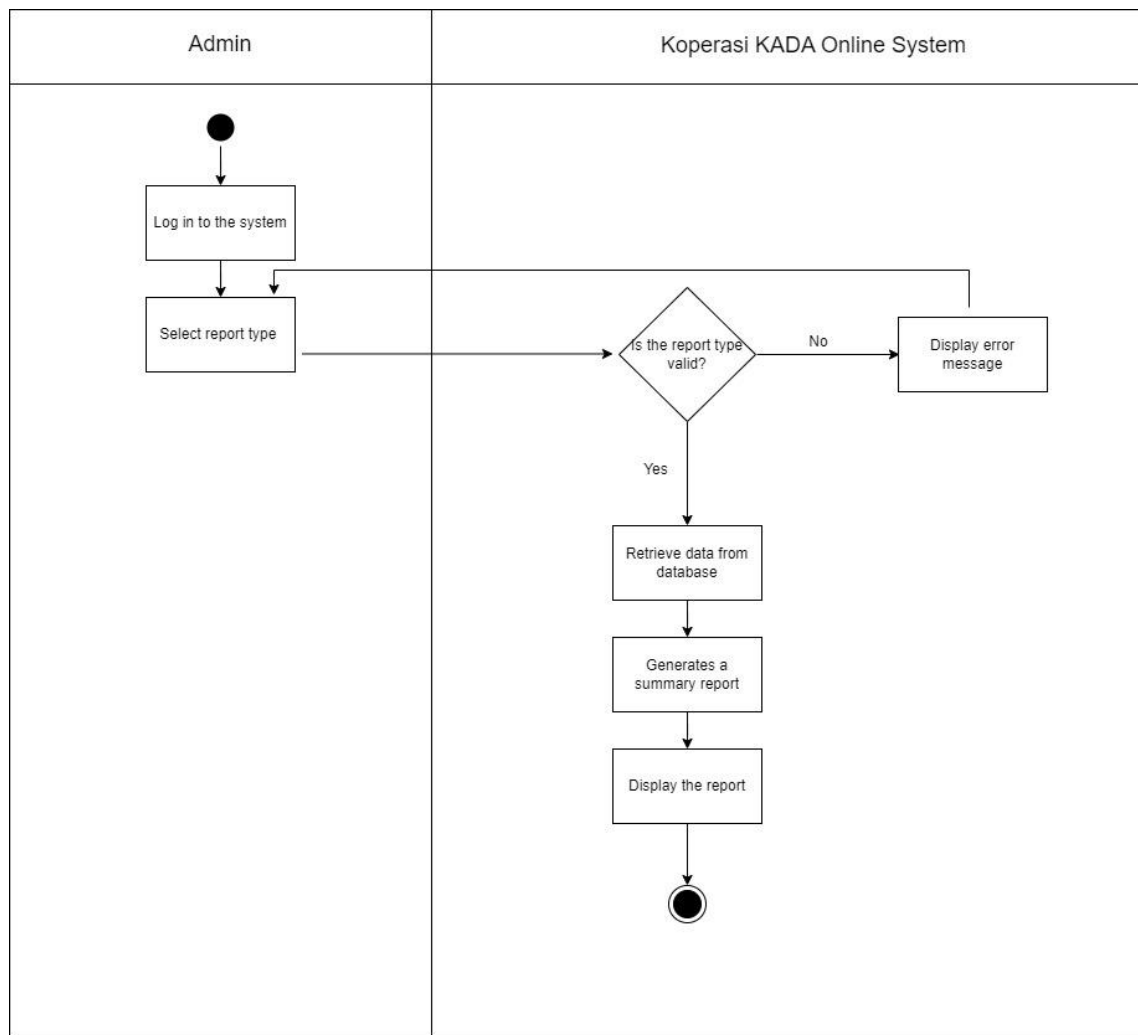


Figure 6.2.7 Activity Diagram for Generate Report

Meeting Log

Meeting #1 (20/1/2025, 8.00pm - 11.00pm)



Our first meeting was on January 20, 2025, at 8 p.m. sharp. The purpose of this meeting is to discuss the report's requirements, which include system introduction, database requirements, global erd, and system implementation. Everyone in the group arrived on time for the meeting, and we discussed the project together.

Meeting #2 (25/1/2025, 9.00pm - 11.00pm)



We held our second meeting on January 25, 2025, at 9:00 p.m. The purpose of this meeting is to conduct a final check on the report that we completed before submitting it. Every group member took part in this discussion, and we checked to confirm if we had completed the report appropriately.